

Architecture Des Ordinateurs ADO 2

Pr Amadou Dahirou GUEYE ADG

UAM

Contact: dahirou.gueye@uam.edu.sn 2023-
2024

Licence 1 MPI & SML Semestre 2

Plan du cours:

Chapitre 1: Systèmes numériques complexes dans VHDL

Chapitre 2: Composants de base d'un ordinateur

Chapitre 3: Architecture des ensembles d'instructions

Chapitre 4: Programmation niveau assembleur

Objectifs du cours:

Ce cours initie les étudiants aux notions de base de l'architecture informatique et, en particulier, aux choix de l'architecture des ensembles d'instruction et à la hiérarchie de la mémoire des systèmes modernes.

- OG 1:** Comprendre le langage VHDL et simuler les comportements des signaux électriques
- OG 2:** Comprendre les composants de base et le fonctionnement des ordinateurs
- OG 3:** Concevoir et mettre en œuvre un processeur au niveau du transfert de registre à l'aide de synthétiseurs logiques et de simulateurs.
- OG 4:** Élaborer des programmes de langage d'assemblage

Chapitre 1:

Langage VHDL

Plan

1. Introduction au VHDL
2. Structure de Base du VHDL
3. Concepts clés (structure alternative, sélection multiple, structure répétitive, fonction, etc.)
4. Simulation
5. Conclusion

Introduction

Historique VHDL

En **1981**, le Département de la Défense (DoD) des Etats-Unis d'Amérique a initié puis dirigé le projet "Very High Speed Integrated Circuit" (**VHSIC**).

Ce projet avait pour but de formaliser la description des circuits intégrés développés pour le DoD dans un langage commun.

Le Développement du langage a été confié par le DoD aux sociétés IBM, Intermetrics et Texas Instruments.

Le langage **VHDL** signifie **V**ery high speed integrated circuit **H**ardware **D**escription **L**anguage a été développé à partir de **1983**.

Puis, il a été normalisé par l'IEEE (Institute of Electrical and Electronic Engineers) ou Institut des ingénieurs électriciens et électroniciens en **1987**.

Introduction au VHDL

VHDL est un langage de description matériel utilisé pour modéliser, simuler et synthétiser des circuits numériques.

Bref, un langage de description matériel destiné à représenter le comportement et l'architecture d'un système électronique numérique

Introduction au VHDL

Il existe d'autre langage comme Verilog qui aussi un langage de description. Verilog et VHDL sont les plus utilisés

Verilog, de son nom complet Verilog HDL est un langage de description matériel de circuits logiques en électronique, utilisé pour la conception d'ASICs (application-specific integrated circuits, circuits spécialisés) et de FPGAs (field-programmable gate array).

Introduction au VHDL

VHDL est un langage précis qui permet de créer des systèmes fonctionnels et réutilisables.

Il est idéal pour simuler et vérifier les conceptions, et il fonctionne bien avec les FPGA pour un prototypage rapide et les ASIC pour une conception optimisée.

NB:

FPGA (Field Programmable Gate Arrays) sont des circuits intégrés dont la fonctionnalité est entièrement programmable par l'utilisateur après fabrication

ASIC (Application specific Integrated Circuits)

Introduction au VHDL

Des logiciels ...

- Le logiciel **ModelSim SE** permet la simulation temporelle au niveau RT (transfert de registre) ou niveau porte, à partir des langages VHDL ou Verilog

- **Quartus II** d'Altera (Intel), **ISE** et **Vivado** (Xilinx).

Tous 4 gratuits dans leur version Web edition.

Structure de Base du VHDL

Structure de Base du VHDL

Architecture du VHDL

Syntaxe de base

Opérations Logiques

Structure de Base du VHDL

La structure de base est définie par un couple :

- Entité
- Architecture

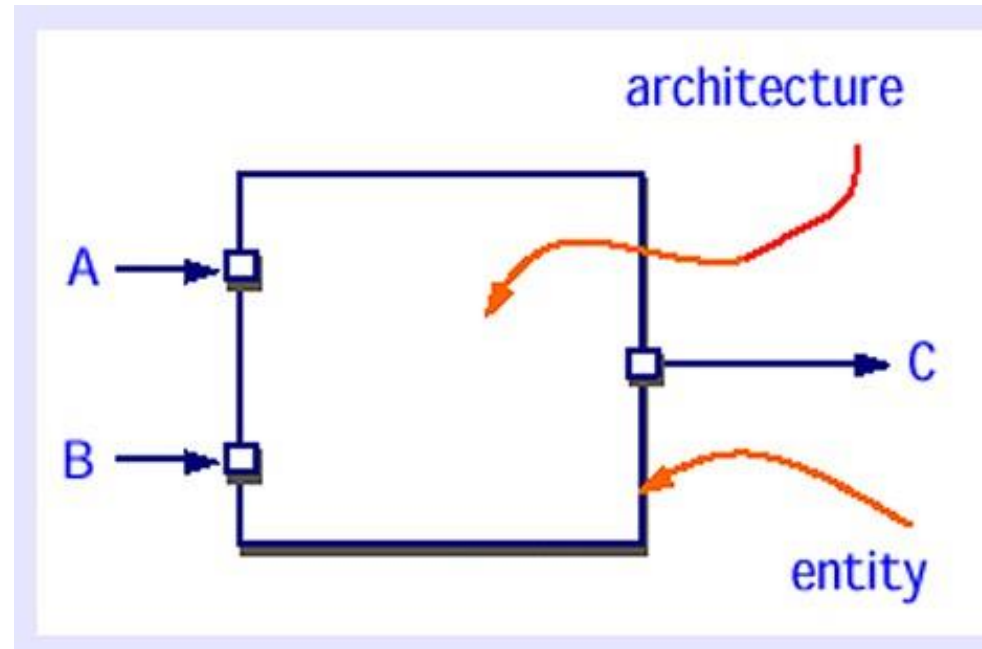
Structure de Base du VHDL

Entité

L'entité définit l'interface d'un composant (vue externe). Elle spécifie les ports (entrées et sorties) et donne un aperçu de ce que fait le composant sans en révéler les détails internes

Structure de Base du VHDL

Entité



Structure de Base du VHDL

Syntaxe pour l'entité:

Entity Nom_Entite **is**

Port (

nom_port1 : in STD_LOGIC; -- Port d'entrée

nom_port2 : out STD_LOGIC -- Port de sortie

);

end Nom_Entite **;**

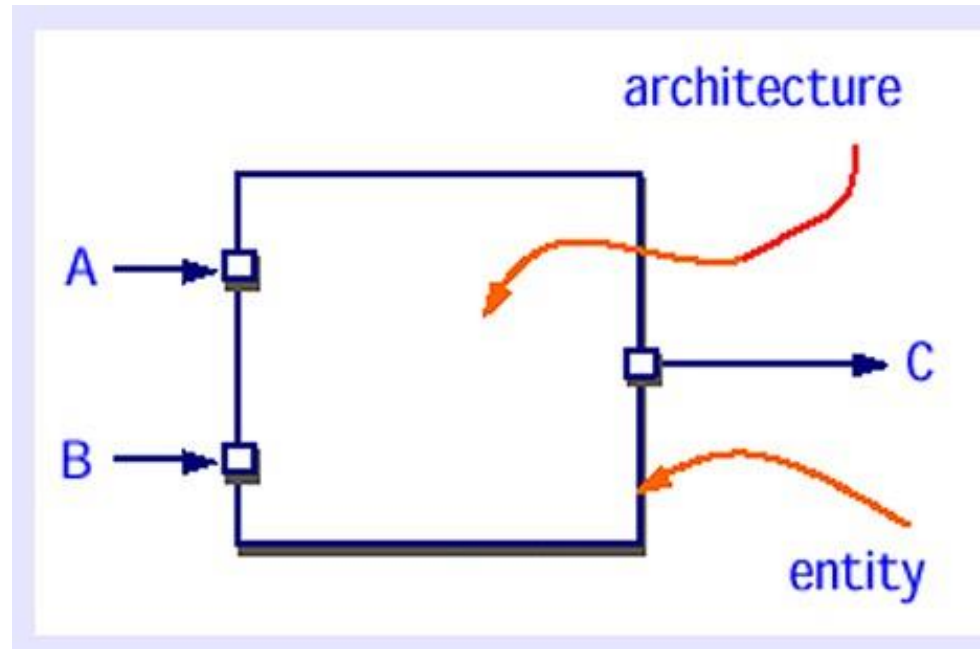
Structure de Base du VHDL

Architecture

L'architecture décrit le comportement ou la structure interne de l'entité. Elle peut contenir des processus, des signaux, des déclarations de variables, et la logique qui réalise la fonction de l'entité.

Structure de Base du VHDL

Architecture



Structure de Base du VHDL

Architecture du VHDL

Syntaxe pour l'architecture

Structure de Base du VHDL

Architecture du VHDL

```
architecture Nom_Architecture of Nom_Entite is
    -- Déclaration des signaux internes
    signal interne_signal : STD_LOGIC;
begin
    -- Logique du circuit
    processus (nom_port1) is
    begin
        interne_signal <= not nom_port1; -- Exemple de logique
        nom_port2 <= interne_signal;
    end processus;
end Nom_Architecture;
```

Structure de Base du VHDL

Architecture du VHDL

Les ports

Les ports sont des points d'entrée et de sortie pour un composant. Ils permettent de connecter l'entité à d'autres entités ou à l'extérieur du circuit.

Il est à noter qu'on peut avoir plusieurs architectures liées à une entité.

Structure de Base du VHDL

Architecture du VHDL

Ports: signaux caractérisés par un mode et un type

4 modes possibles:

in : Port d'entrée, reçoit des données.

out : Port de sortie, envoie des données.

inout : Port bidirectionnel, peut à la fois recevoir et envoyer des données.

buffer: Pour les signaux de sortie réentrants

2 types de base:

Scalaire: STD_LOGIC

Vectorel: STD_LOGIC_VECTOR

Structure de Base du VHDL

Architecture du VHDL

Les signaux

Les signaux sont utilisés pour transporter des données entre différentes parties de l'architecture. Ils représentent des connexions internes au circuit et peuvent être utilisés pour interconnecter des processus ou des composants.

Structure de Base du VHDL

Architecture du VHDL

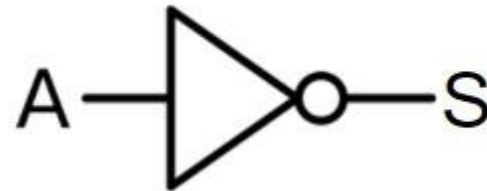
Syntaxe de déclaration d'un signal

signal nom_signal : **STD_LOGIC**; -- Déclaration d'un signal de type logique

Structure de Base du VHDL

Architecture du VHDL

Inverseur logique



Structure de Base du VHDL

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
-- Entité d'un inverseur logique  
entity Inverseur is  
    Port (  
        A : in STD_LOGIC;    -- Port d'entrée  
        S : out STD_LOGIC    -- Port de sortie  
    );  
end Inverseur;
```

Structure de Base du VHDL

-- Architecture du inverseur

architecture Comportement **of** Inverseur **is**
begin

-- Processus qui réalise l'inversion

process(A) **is**
begin

S <= not A; -- Inversion du signal d'entrée

end process;

end Comportement;

Structure d'un programme VHDL

Sur un exemple simple : la description d'une porte **ET**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity porteET is
    Port ( entree1 : in STD_LOGIC;
          entree2 : in STD_LOGIC;
          sortie  : out STD_LOGIC);
end porteET;

architecture Behavioral of porteET is
begin
    sortie <= entree1 and entree2;
end Behavioral;
```

Architecture interne

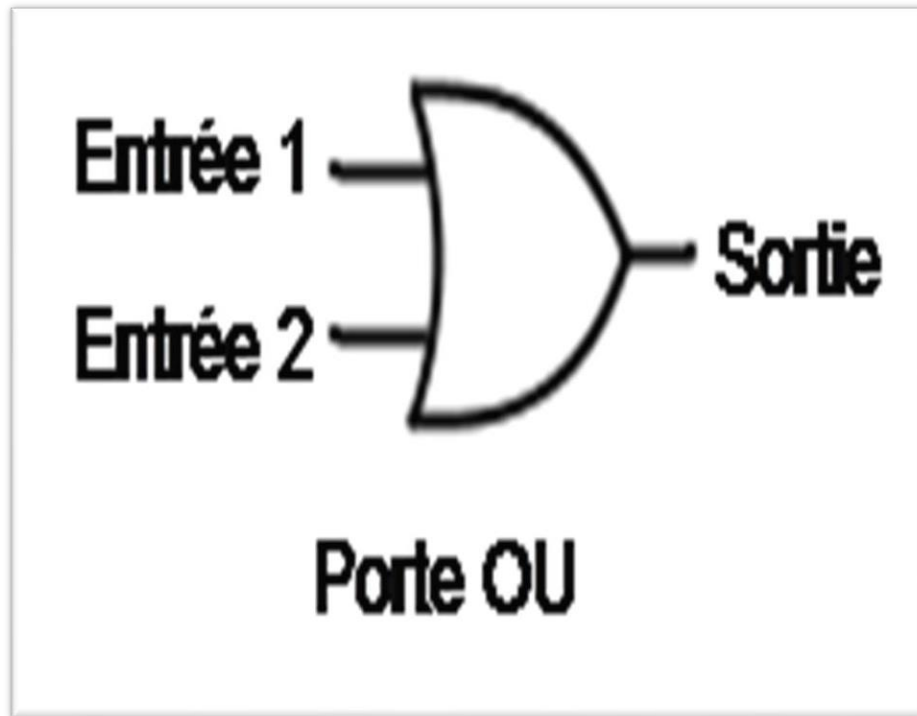
- Description interne de haut niveau et portable
 - Comportementale
 - Structures de traitements de flots de données.
 - Processus séquentiels.
 - Structurale hiérarchique
 - Instanciation de blocs décrits en VHDL et description de leurs interconnexions.
- Description interne de bas niveau (structurale) à la portabilité limitée
 - Description structurale reposant sur l'instanciation de primitives du composant ciblé.
- Syntaxe (description comportementale seulement)

```
architecture nom de l'architecture of nom de l'entité is
    -- Signaux et constantes internes
    -- Fonctions internes
begin
    -- Traitements de flot de données
    -- Descriptions séquentielles
end nom de l'architecture;
```



Structure d'un programme VHDL

Sur un exemple simple : la description d'une porte **OU(OR)**

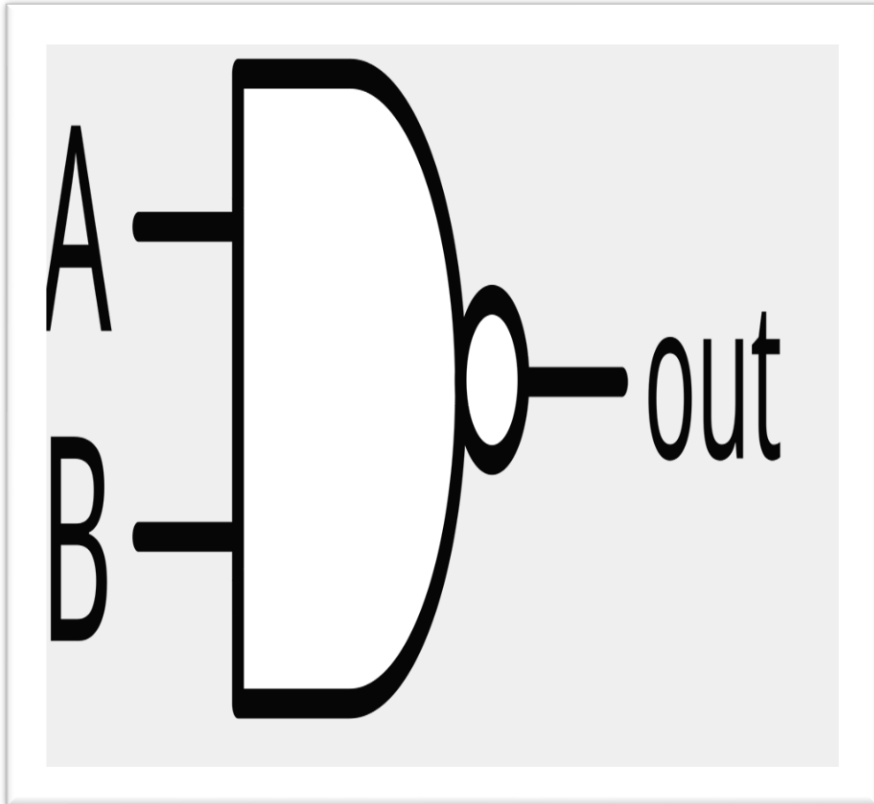


```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
-- Déclaration de l'entité
entity Porte_OR is
    Port (
        A : in STD_LOGIC; -- Entrée A
        B : in STD_LOGIC; -- Entrée B
        Y : out STD_LOGIC -- Sortie Y
    );
end Porte_OR;
-- Déclaration de l'architecture
architecture Comportemental of Porte_OR is
begin
    -- La sortie Y est l'opération OU logique des entrées A et B
    Y <= A or B;
end Comportemental;
```



Structure d'un programme VHDL

Sur un exemple simple : la description d'une porte **NON ET (NAND)**



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

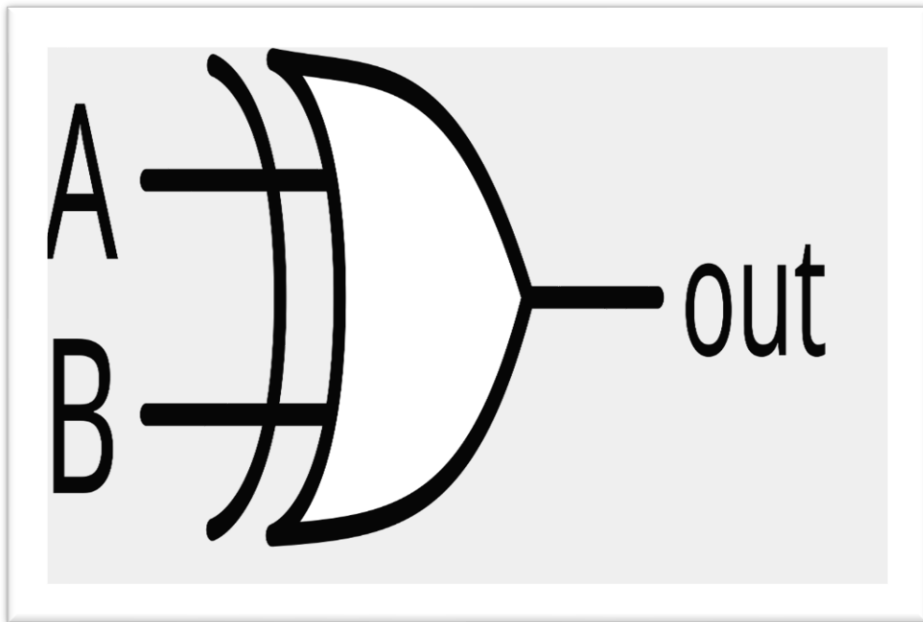
-- Déclaration de l'entité
entity Porte_NAND is
    Port (
        A : in STD_LOGIC; -- Entrée A
        B : in STD_LOGIC; -- Entrée B
        Y : out STD_LOGIC -- Sortie Y
    );
end Porte_NAND;

-- Déclaration de l'architecture
architecture Comportemental of Porte_NAND is
begin
    -- La sortie Y est l'opération NAND logique des entrées A et B
    Y <= not (A and B);
end Comportemental;
```



Structure d'un programme VHDL

Sur un exemple simple : la description d'une porte **XOR**



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

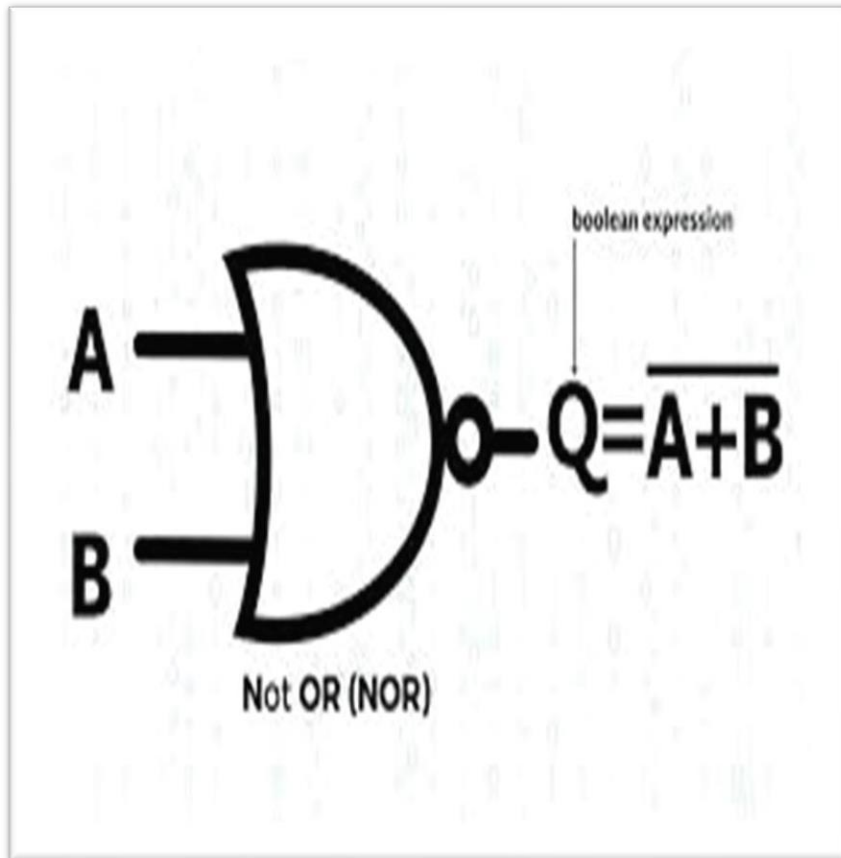
-- Déclaration de l'entité
entity Porte_XOR is
    Port (
        A : in STD_LOGIC; -- Entrée A
        B : in STD_LOGIC; -- Entrée B
        Y : out STD_LOGIC -- Sortie Y
    );
end Porte_XOR;

-- Déclaration de l'architecture
architecture Comportemental of Porte_XOR is
begin
    -- La sortie Y est l'opération XOR logique des entrées A et B
    Y <= A xor B;
end Comportemental;
```



Structure d'un programme VHDL

Sur un exemple simple : la description d'une porte **NON OU (NOR)**



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Déclaration de l'entité
entity Porte_NOR is
    Port (
        A : in STD_LOGIC; -- Entrée A
        B : in STD_LOGIC; -- Entrée B
        Y : out STD_LOGIC -- Sortie Y
    );
end Porte_NOR;

-- Déclaration de l'architecture
architecture Comportemental of Porte_NOR is
begin
    -- La sortie Y est l'opération NOR logique des entrées A et B
    Y <= not (A or B);
end Comportemental;
```



Structure de Base du VHDL

Architecture du VHDL

Syntaxe de base

Opérations Logiques

Structure de Base du VHDL

Syntaxe de base

Type de données

bit : Un bit logique représentant 0 ou 1.

std_logic : Un type pour la logique standard qui peut avoir plusieurs états comme '0', '1', 'Z' (haute impédance), 'W' (attente), etc.

NB: L'impédance électrique est mesurée en Ohms et représente la résistance totale que présente le câble au courant électrique qui le traverse. Rapport U/I

Structure de Base du VHDL

Syntaxe de base

Type de données

integer : Pour les nombres entiers.

real : Pour les nombres réels.

boolean : Pour les valeurs booléennes (TRUE ou FALSE).

character : Pour les caractères ASCII.

Structure de Base du VHDL

Syntaxe de base

Déclaration des variables

La syntaxe pour déclarer une variable en VHDL est la suivante :

variable nom_de_variable : type_de_donnée := valeur_initiale;

Structure de Base du VHDL

Syntaxe de base

Déclaration des constantes

Les constantes sont des valeurs fixes qui ne peuvent pas être modifiées après leur déclaration. Elles sont utilisées pour définir des valeurs qui ne changent jamais tout au long de la simulation.

Structure de Base du VHDL

Syntaxe de base

Déclaration des constantes

Syntaxe:

```
constant nom_constante : type_de_donnée := valeur;
```

Structure de Base du VHDL

Opérations logiques

Les opérateurs logiques sont utilisés pour effectuer des opérations de base sur des bits ou des signaux de type `std_logic` ou `bit`.

Structure de Base du VHDL

Opérations logiques

Opérateurs	Description	Exemple
and	Opérateur ET logique	a and b
or	Opérateur OU logique	a or b
not	Négation logique (NON)	not a
nand	Opérateur NAND logique	a nand b
xor	Ou exclusif	a xor b

Structure de Base du VHDL

Opérations logiques

Exemple:

```
signal a, b, c : std_logic;
```

```
process (a, b)
```

```
begin
```

```
    c <= a and b;      -- c est '1' seulement si a et b  
    sont tous les deux '1'  
end process;
```

Structure de Base du VHDL

Opérations Arithmétiques

Les opérateurs arithmétiques permettent d'effectuer des calculs sur des valeurs numériques. Ces opérateurs fonctionnent généralement avec les types **integer**, **unsigned**, **signed**, et **std_logic_vector** (avec des conversions appropriées).

Structure de Base du VHDL

Opérations arithmétiques

Opérateurs	Description	Exemple
+	Addition	$a+b$
-	Soustraction	$a-b$
*	Multiplication	$a*b$
/	Division	a/b
mod	Reste de la division (modulo)	$a \bmod b$
rem	Reste de la division entière	$a \text{ rem } b$
abs	Valeur absolue	$\text{abs } a$

Structure de Base du VHDL

Opérations Arithmétiques

Exemple d'opérations arithmétiques :

```
signal a, b : integer := 0;  
signal result : integer;  
  
process (a, b)  
begin  
    result <= a + b;    -- Addition de a et b  
end process;
```

Structure de Base du VHDL

Assignment

Les opérateurs $:=$ et $<=$ sont utilisés dans des contextes différents.

L'opérateur $:=$ est utilisé pour affecter une valeur à une variable. La mise à jour des variables se fait immédiatement après l'affectation, c'est-à-dire que leur nouvelle valeur est disponible instantanément à l'intérieur du même processus ou bloc de code.

Structure de Base du VHDL

Assignment

L'opérateur $\leq$ est utilisé pour affecter une valeur à un signal. Contrairement aux variables, les signaux sont mis à jour avec un retard, généralement au prochain cycle d'horloge.

Structure de Base du VHDL

Assignment conditionnelle

L'assignation conditionnelle permet d'affecter une valeur à un signal selon une ou plusieurs conditions. Il existe deux principales formes d'assignation conditionnelle en VHDL : **when-else** et **with-select-when**.

Structure de Base du VHDL

Assignment conditionnelle

Assignment conditionnelle avec when-else

Cette forme est utilisée pour affecter des valeurs à un signal en fonction de conditions logiques. Elle est utile pour les sélections simples ou lorsque seulement deux options sont possibles.

Structure de Base du VHDL

Assignment conditionnelle

Syntaxe:

```
signal_nom <= valeur1 when condition else  
valeur2;
```

Structure de Base du VHDL

Assignment conditionnelle

Exemple d'assignation conditionnelle when-else :

```
signal a, b, c : std_logic;
```

```
process (a, b)
```

```
begin
```

```
    c <= '1' when (a = '1' and b = '1') else '0';
```

```
    -- c est '1' seulement si a et b sont tous les deux '1'
```

```
end process;
```

Structure de Base du VHDL

Assignment conditionnelle

Assignment conditionnelle avec with-select-when

Cette forme est utilisée pour des sélections multiples (semblable à une instruction case). Elle est utile lorsque vous avez plusieurs choix possibles à partir d'une expression de sélection

Structure de Base du VHDL

Assignment conditionnelle

Assignment conditionnelle avec with-select-when **Syntaxe:**

```
with expression select  
    signal_nom <= valeur1 when choix1,  
                  valeur2 when choix2,  
                  valeur_par_défaut when others;
```

Structure de Base du VHDL

Assignment conditionnelle

Exemple d'assignation conditionnelle with-select-when :

```
signal a, b, c : std_logic_vector(1 downto 0);
signal result  : std_logic;
process (a, b)
begin
    with a select
        result <= '1' when "00",
                  '0' when "01",
                  '1' when "10",
                  '0' when others;
end process;
```

Structure de Base du VHDL

Assignment conditionnelle

Exercice d'application:

Voici un exemple plus complet qui combine des opérations logiques, arithmétiques et une assignation conditionnelle :

Structure de Base du VHDL

Assignment conditionnelle

Exercice d'application:

```
entity Example is
  Port (
    a, b      : in  std_logic;    -- Entrées Logiques
    sel       : in  std_logic;    -- Sélection de l'opération
    x, y      : in  integer;      -- Entrées numériques
    result    : out integer;      -- Résultat de l'opération arithmétique
    logic_out : out std_logic     -- Résultat de l'opération logique
  );
end Example;
```


Structure de Base du VHDL

Assignment conditionnelle

Exercice d'application:

```
architecture Comportement of Example is
begin
    process (a, b, sel, x, y)
    begin
        -- Opération arithmétique : addition ou soustraction selon 'sel'
        result := x + y when sel = '1' else x - y;

        -- Opération logique : AND ou OR selon 'sel'
        logic_out <= (a and b) when sel = '1' else (a or b);
    end process;
end Comportement;
```

Modélisation Comportementale et Structurelle

Modélisation Comportementale et Structurelle

Modélisation comportementale

Elle décrit le fonctionnement d'un circuit sans se soucier de la manière dont les composants internes sont connectés. On se concentre uniquement sur ce que fait le circuit en termes d'algorithmes ou de logique séquentielle/combinatoire

Modélisation Comportementale et Structurelle

Modélisation comportementale

Approche de haut niveau : Le comportement est défini en utilisant des instructions de haut niveau (comme des conditions, des boucles, des opérations arithmétiques) pour exprimer ce que le circuit doit faire.

Modélisation Comportementale et Structurelle

Modélisation comportementale

Exemple de Modélisation Comportementale :
Additionneur Simple

Modélisation Comportementale et Structurelle

Modélisation structurelle

Définition : Elle décrit un circuit en termes de composants individuels et de la manière dont ils sont interconnectés pour réaliser une fonction particulière. Ici, la structure interne du circuit est explicitement définie.

Modélisation Comportementale et Structurelle

Modélisation structurelle

Approche bas niveau : On décrit le circuit comme un ensemble de portes logiques ou de sous-modules reliés entre eux.

Modélisation Comportementale et Structurelle

Modélisation structurelle

Exemple de Modélisation Comportementale :
Additionneur Simple

Boucles et Conditions

En VHDL, les boucles et les structures conditionnelles permettent de contrôler le flux du programme et de répéter des instructions. Voici comment fonctionnent les boucles **for** et **while** ainsi que les structures **if** et **case** en VHDL.

Boucles et Conditions

Boucle **for**

La boucle **for** permet de répéter une suite d'instructions un nombre fixe de fois.

La variable de la boucle est locale au bloc et ne peut pas être modifiée dans la boucle.

Boucles et Conditions

Boucle for

Exemple de boucle for :

```
process
    variable i : integer;
begin
    for i in 0 to 7 loop
        -- Code à exécuter 8 fois (de i = 0 à i = 7)
    end loop;
end process;
```

Boucles et Conditions

Boucle while

Exemple de boucle while :

```
process
    variable i : integer := 0;
begin
    while i < 5 loop
        -- Code à exécuter tant que i < 5
        i := i + 1;  -- Incrémentation pour éviter une boucle infinie
    end loop;
end process;
```

Boucles et Conditions

Structure if

La structure **if** permet d'exécuter une section de code si une condition est vraie. On peut aussi inclure des clauses **elsif** et **else**.

Boucles et Conditions

Structure if

Exemple de structure if :

```
process (a, b)
begin
    if a = '1' and b = '0' then
        -- Code si la condition est vraie
    elsif a = '0' and b = '1' then
        -- Code si cette autre condition est vraie
    else
        -- Code sinon
    end if;
end process;
```

Boucles et Conditions

Structure case

La structure **case** est une alternative au if lorsqu'on doit vérifier plusieurs valeurs d'une même variable.

Chaque choix est mutuellement exclusif et nécessite une clause **when others** pour couvrir tous les cas non spécifiés.

Boucles et Conditions

Structure if

Exemple de structure case :

```
process (sel)
begin
    case sel is
        when "00" =>
            -- Code si sel = "00"
        when "01" =>
            -- Code si sel = "01"
        when "10" =>
            -- Code si sel = "10"
        when others =>
            -- Code pour les autres valeurs de sel
    end case;
end process;
```


Boucles et Conditions

Résumé des Différences

Structure	Usage	Particularité
for	Répétition fixe	Variable boucle locale
while	Répétition conditionnelle	Moins utilisé pour la synthèse
if	Conditionnel simple ou multi-conditions	Utilisé pour des tests logiques
case	Tests sur une variable avec plusieurs valeurs possibles	Nécessite when others pour les cas restants

Simulation

La simulation est le processus de vérification du comportement du code VHDL pour s'assurer qu'il fonctionne comme prévu avant de passer à la synthèse. Elle permet de détecter et corriger les erreurs de logique ou de conception.

Simulation

Outils de simulation

ModelSim : Logiciel de simulation largement utilisé pour tester le code VHDL. Il permet de créer des stimuli, d'observer les signaux et de vérifier le comportement du circuit en conditions simulées.

ModelSim

Simulation

Outils de simulation

GHDL : Simulateur open-source de VHDL, particulièrement utile pour les simulations rapides et les projets académiques.



Simulation

Téléchargement de ModelSim

Pour télécharger **ModelSim**, il faut aller dans le lien de téléchargement suivant :

https://downloads.intel.com/akdlm/software/acdsinst/18.1std/625/ib_installers/ModelSimSetup-18.1.0.625-windows.exe

Simulation

2 Cas Pratiques pour la simulation:

- en forçant les valeurs d'entrée
- avec un banc de test

Simulation

Cas Pratique 1: en forçant les valeurs d'entrée :

Cas de l'Inverseur: (TP à réaliser par chaque étudiant)

Démarche:

1. Installer ModelSim
2. Créer un projet
3. Créer un fichier un fichier inverseur.vhd dans le projet
4. Compiler le fichier
5. Démarrer Simulate en sélectionnant le fichier à partir du work
6. Visualiser le résultat de la simulation en créant un environnement wave à partir du fichier (clic droit puis add to wave)
7. Entrez (forcing) des valeurs d'entrées
8. Exécuter
9. On peut aussi faire des tests à des temps déterminés (ex. 10 ps)

Simulation

Cas Pratique: *Banc de test (Testbench)*

Le **testbench** applique différents stimuli (valeurs logiques) à l'entrée a et observe la sortie s.

■ *Fichier : test_invervseur.vhd*

```
LIBRARY ieee;  
USE ieee.std_logic_1164.ALL;  
-- Définition d'une entité vide  
ENTITY test_Inverseur IS  
END test_Inverseur;
```


Simulation

Banc de test (Testbench)

```
ARCHITECTURE Comportement OF test_Inverseur IS
```

```
-- Composant à tester
```

```
COMPONENT Inverseur
```

```
PORT(
```

```
    A : IN std_logic;
```

```
    S : OUT std_logic
```

```
);
```

```
END COMPONENT;
```

```
-- Signaux de test
```

```
signal a : std_logic := '0';
```

```
signal s : std_logic;
```

Simulation

Banc de test (Testbench)

BEGIN

-- *Instanciación de l'inverseur*

uut: Inverseur PORT MAP (

 A => a,

 S => s

);

-- *Processus pour appliquer les stimuli*

stim_proc: process

Simulation

Banc de test (Testbench)

```
begin
    -- Cas 1 : a=0, y devrait être 1
    a <= '0';
    wait for 10 ns;
    report "sortie s=1, teste avec succès" ;

    -- Cas 2 : a=1, y devrait être 0
    a <= '1';
    wait for 10 ns;

    wait; -- Arrêt du processus
end process;
END Comportement;
```

Simulation

Banc de test (Testbench)

Explication:

L'entité **test_inverseur** est définie sans ports, car il s'agit d'un banc de test, et elle n'a pas besoin d'entrées ni de sorties externes.

L'architecture **Comportement** décrit le comportement de ce banc de test.

Simulation

Banc de test (Testbench)

Ici, le composant **Inverseur** est déclaré avec un port d'entrée a et une sortie S. Ce composant représente la porte NOT que nous voulons tester.

Cas 1 : Le signal a est mis à '0', et après 10 ns, la sortie y devrait être '1' (car une porte NOT inverse l'entrée).

Cas 2 : Le signal a est mis à '1', et après 10 ns, la sortie y devrait être '0'. Le `wait; final` arrête le processus, car il n'y a pas d'autres stimuli à appliquer.

Simulation

Importance de la simulation

Avant la synthèse, la simulation permet de :

- *Valider la logique du code*
- *Détecter et corriger les erreurs fonctionnelles,*
- *Optimiser le code VHDL*

La synthèse

Définition

La synthèse transforme le code VHDL en un schéma matériel qui sera implémenté sur une puce, soit FPGA (Field-Programmable Gate Array), soit ASIC (Application-Specific Integrated Circuit).

La sythèse

Puce FPGA



Cyclone FPGA, Altera

La synthèse

Rôle de la synthèse

Pour les FPGAs : La synthèse convertit le code VHDL en une configuration spécifique des blocs logiques et interconnexions du FPGA. Ces blocs logiques incluent des LUTs (Look-Up Tables), des registres, et des blocs de mémoire.

La synthèse

Rôle de la synthèse

Pour les ASICs : La synthèse crée un circuit fixe qui sera gravé dans le silicium. Contrairement aux FPGAs, l'ASIC ne peut être reprogrammé, ce qui rend la synthèse d'autant plus critique.

La synthèse

Rôle de la synthèse

Les outils de synthèse sont :

- Vivado
- Quartus
- Synopsys

Cas Pratique d'un banc de test

FIN Chapitre 1

Chapitre 2:

Composants de base d'un ordinateur

Plan Chapitre 2

1. Introduction aux composants
2. Unité de traitement
3. La mémoire
4. Les supports de stockage
5. La carte mère
6. Le bus
7. Les périphériques
8. Les systèmes d'exploitations
9. Conclusion

Introduction aux composants de l'ordinateur

Ordinateur

Un ordinateur de bureau est conçu pour être toujours au même endroit, généralement sur un bureau. Il se compose :

Ordinateur



Ordinateur

1. d'une "unité centrale", appelée aussi "tour". Celle-ci contient les principaux composants de l'ordinateur. C'est également sur celle-ci que vous trouverez le bouton pour allumer l'ordinateur :
2. d'un écran : qui permet d'afficher le contenu de l'ordinateur.

Ordinateur

- 3. d'un clavier : qui permet de communiquer avec l'ordinateur en tapant du texte.
- 4. d'une souris : qui permet de déplacer le curseur à l'écran.

Ordinateurs "tout-en-un"

Il existe également des ordinateurs sur lesquels tous les composants sont regroupés derrière l'écran, on les appelle ordinateurs "tout-en-un".

Ordinateurs "tout-en-un" All in One



Ordinateurs "tout-en-un"

À ces ordinateurs, il faut ajouter un clavier et une souris pour pouvoir les utiliser.

Les sept meilleurs ordinateurs tout en un en 2024

1. iMac (24 pouces, 2021) - Le meilleur ordinateur tout-en-un à l'heure actuelle.
2. iMac (27 pouces, 2020) - Un formidable ordinateur tout-en-un parfait pour l'édition photo et vidéo.
3. HP Envy 34 All-in-One - Une alternative sérieuse à l'iMac pour les professionnels de la création numérique.
4. Lenovo IdeaCentre AIO 3 Gen 6
5. Acer Aspire C24-1700 Ordinateur
6. HP All-in-One 24-cb1002ss Bundle PC
7. Saintdise Ordinateur de Bureau Tout-en-Un

Ordinateurs portables

Les ordinateurs portables sont conçus pour être compacts et mobiles. Ainsi, ils ne disposent pas d'une "tour", tous les composants se situent sous le clavier.

Ordinateurs portables



Ordinateurs portables

Sur ce type d'ordinateur, brancher une souris n'est pas obligatoire, car il y a un "touchpad" : une surface sensible au toucher qui permet de déplacer le pointeur à l'écran.

Ordinateurs portables

Comme pour les ordinateurs de bureau, d'autres périphériques peuvent être ajoutés.

Tablettes

Il existe aussi des "tablettes", ce sont des ordinateurs mobiles à la taille réduite. Les écrans sont tactiles et affichent un clavier virtuel. Tous les composants sont situés derrière l'écran.

Tablettes



Tablettes

Ainsi, les tablettes n'ont pas besoin d'unité centrale, ni de clavier ou de souris. Sur certains modèles, il est toutefois possible d'y connecter un clavier et/ou une souris

Unité de traitement

Les composants de l'ordinateur

Processeur

C'est le cerveau de l'ordinateur.

Il réalise tous les calculs nécessaires au fonctionnement de l'ordinateur.

Les composants de l'ordinateur:

Unité de mesure du Processeur

C'est notamment la fréquence du processeur, c'est-à-dire la vitesse à laquelle il travaille, qui détermine la rapidité de votre ordinateur.

Cette fréquence s'exprime en Giga Hertz (GHz).

La fréquence a une relation avec le temps d'exécution ou traitement: $F=1/T$

Les composants de l'ordinateur

Processeur

- Dual Core = 2 cœurs

J'ai deux cœurs (chaque cœur traite une opération en une seconde), moi j'ai deux opérations à traiter

- Quad Core = 4 cœurs
- Hexa Core = 6 cœurs
- Octa Core = 8 cœurs

Les composants de l'ordinateur

Fabricants Processeur

Il existe différents modèles de processeurs et cette technologie évolue rapidement.

Les processeurs des marques **AMD** et **Intel** sont les plus fréquemment rencontrés.

Autres fabricants:

- Motorola
- Dell
- Acer
- IBM
- Toshiba

Les composants de l'ordinateur

Processeur



Les composants de l'ordinateur

Génération Processeur

Dans la marque Intel par exemple, on voit généralement les modèles :

- Intel Core i3 qui correspond à l'entrée (bas) de gamme des processeurs Intel Core
- Intel Core i5 qui constitue le milieu de gamme
- Intel Core i7 et Intel Core i9 qui sont de la gamme supérieure

Actuellement 2024, on est à la 13^{ème} génération

Les composants de l'ordinateur

Génération Processeur

Chez AMD par contre, il est plus difficile de comparer les processeurs

Ryzen 3

Ryzen 5

Ryzen 7

Les composants de l'ordinateur

La carte graphique ou vidéo



Les composants de l'ordinateur

La carte graphique ou vidéo

Elle permet de **produire une image** affichable sur un écran d'ordinateur.

Les composants de l'ordinateur

La carte graphique ou vidéo

La carte graphique peut être intégrée à la carte mère ou dédiée, c'est-à-dire qu'elle est séparée et qu'elle dispose de sa propre mémoire vive. Une carte graphique dédiée est plus puissante, mais coûte plus cher. Elle permet de faire fonctionner correctement des jeux en 3D, des logiciels de retouche vidéo/photo par exemple.

La mémoire

Les composants de l'ordinateur

Ram (Random Access Memory)



Les composants de l'ordinateur

Ram (Random Access Memory)

C'est la mémoire temporaire de l'ordinateur, c'est là que sont stockés tous les fichiers sur lesquels l'utilisateur est en train de travailler.

Autrement dit c'est la RAM qui stocke tous les programmes en cours d'exécution

Cette mémoire est temporaire, car les informations sont supprimées lors de l'arrêt de l'ordinateur.

Les composants de l'ordinateur

Ram (Random Access Memory)

Plus cette mémoire est importante, plus l'ordinateur travaille facilement et rapidement et plus il peut gérer des tâches différentes.

Les composants de l'ordinateur

Ram (Random Access Memory)

La capacité de cette mémoire s'exprime en "Gigaoctets (Go)".

La mémoire vive se présente sous forme de petites barrettes que l'on insère dans la carte mère.

Un ordinateur peut ainsi comporter plusieurs barrettes pour avoir au total (cela dépend des ordinateurs) 4 - 8 - 16 ou 32 Go de RAM.

Seulement des RAM de puissance de 2

RQ: On peut augmenter la RAM en fonction du fabricant

Les composants de l'ordinateur

Ram (Random Access Memory)

Actuellement, les ordinateurs vendus ont souvent une mémoire RAM de 4 gigas. C'est suffisant pour une utilisation basique. Pour jouer à des jeux puissants ou faire de la retouche photo/vidéo, 8 (et même parfois 16) gigas sont nécessaires pour faire tourner correctement un ordinateur.

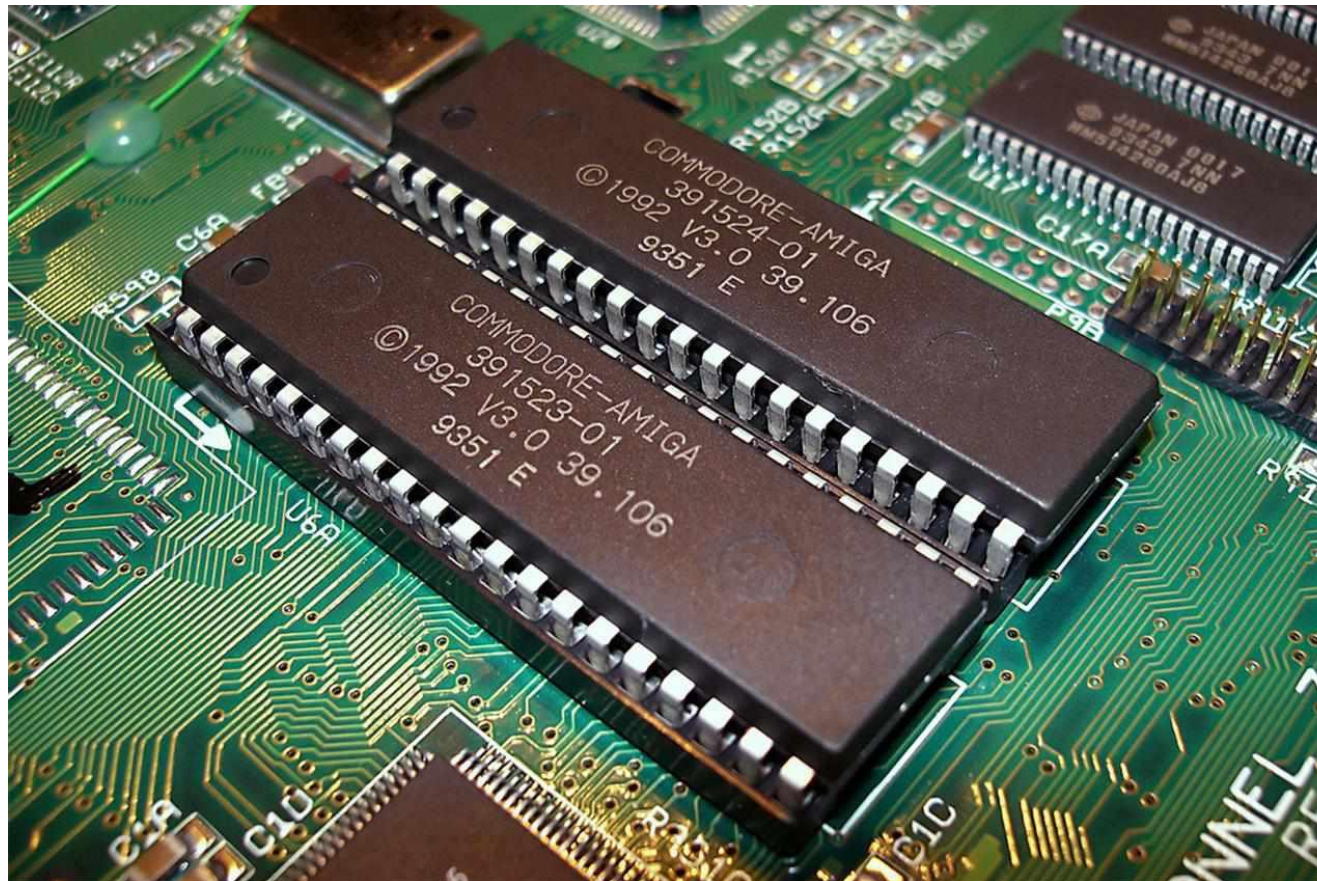
Les composants de l'ordinateur

Mémoire morte (ROM)

La mémoire morte (en anglais ROM = Read Only Memory) est une mémoire en lecture seule, appelée aussi mémoire non volatile, c'est-à-dire une mémoire qui ne s'efface pas à l'extinction de l'ordinateur. Elle stocke le programme de base pour démarrer et utiliser un ordinateur (le BIOS : Basic Input Output System).

Les composants de l'ordinateur

Mémoire morte (ROM)



Les composants de l'ordinateur

Le disque dur



Les composants de l'ordinateur

Le disque dur

C'est le support sur lequel on peut stocker des informations. Les capacités de stockage ne cessent d'augmenter et permettent donc d'enregistrer un grand nombre de données : documents, images, films...

Les composants de l'ordinateur

Le disque dur

Il y a actuellement deux types de disques durs : SSD et HDD. Les SSD ont l'avantage d'être extrêmement rapides, mais ils sont plus chers et de capacité limitée.

Les composants de l'ordinateur

Le disque dur

Il est tout à fait possible d'avoir un ordinateur avec ces 2 types de disques durs, un disque SSD pour le système d'exploitation et un disque HDD pour les données personnelles.

Les supports de stockage

Les composants de l'ordinateur

Les types de stockage

Il existe différents types de stockage:

Disque dur (HDD)

SSD (Solid State Drive) :

NVMe

Les composants de l'ordinateur

Les types de stockage



Disque dur (HDD)

Les composants de l'ordinateur

Les types de stockage



SSD

Les composants de l'ordinateur

Les types de stockage



NVMe

Les composants de l'ordinateur

Les types de stockage

Type de stockage	Description
HDD	Stockage magnétique avec pièces mobiles, offre une grande capacité à bas coût.
SDD	Stockage rapide en mémoire flash sans pièces mobiles, plus résistant aux chocs.
NVMe	Interface ultra-rapide pour SSD, connectée directement au processeur via PCIe.

Le bus

Le bus

Un bus est une structure d'interconnexion, ou ensemble de lignes de communication raccordant plusieurs circuits ou unités fonctionnelles d'un ordinateur.

Le bus

Les principaux bus que l'on trouve sont des bus de données, des bus d'adresse et des bus de commande.

La carte mère

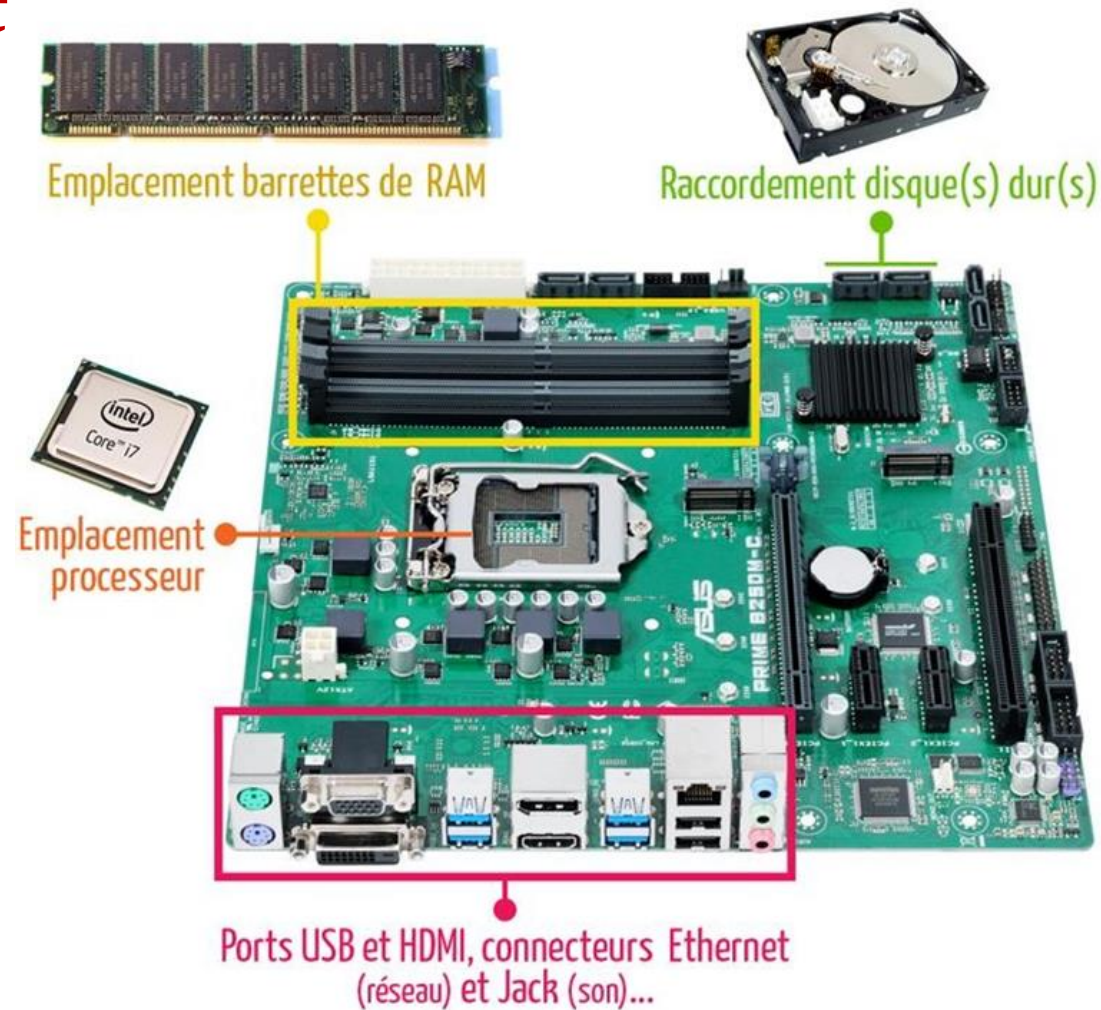
Les composants de l'ordinateur

La carte mère

- Elle se trouve au centre de l'ordinateur et connecte tous les composants de l'ordinateur.
- La carte mère contient les connexions pour le processeur, la mémoire, les unités de stockage...
- Elle intègre une carte son et une carte graphique.

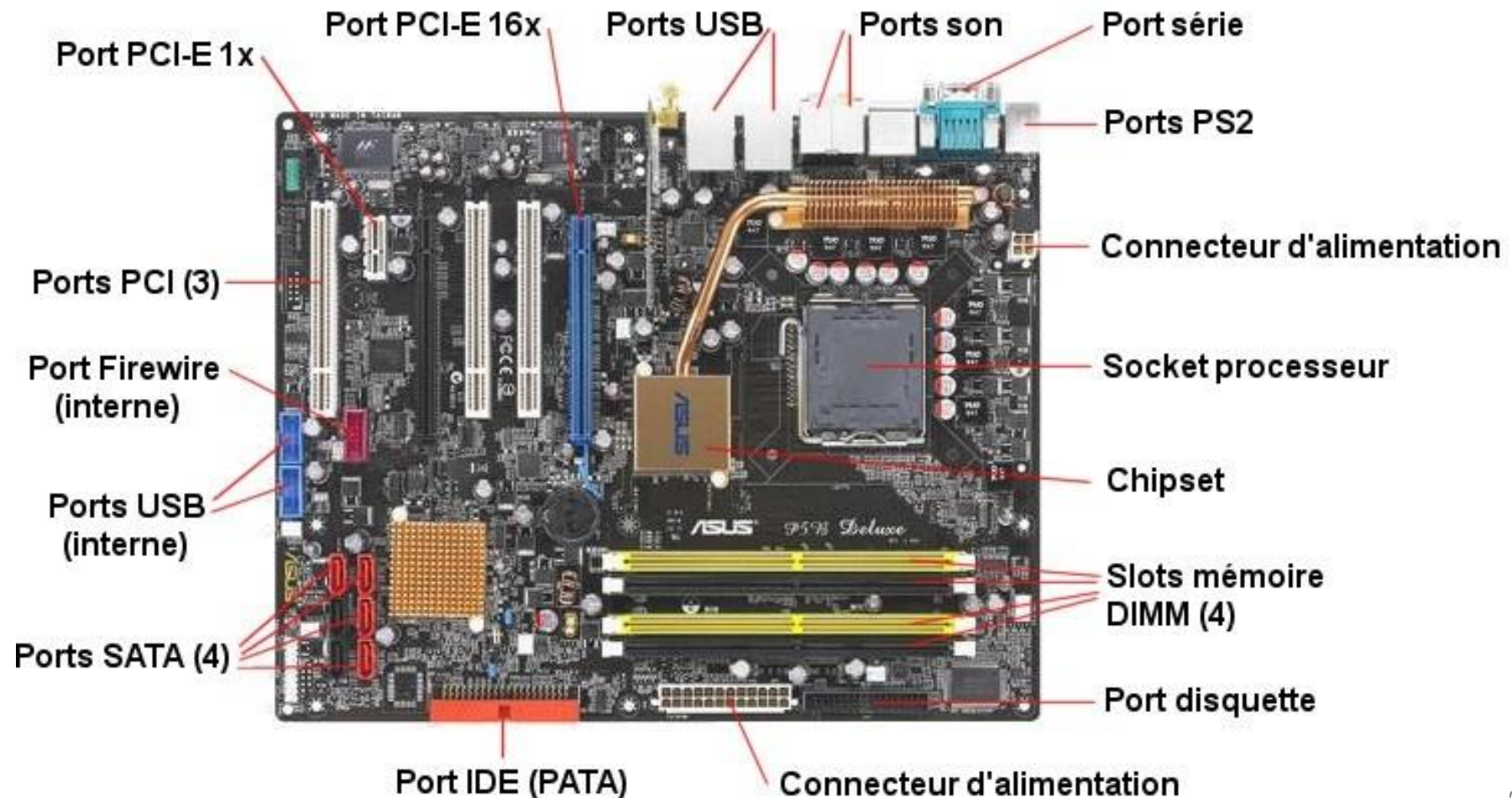
Les composants de l'ordinateur

La carte mère



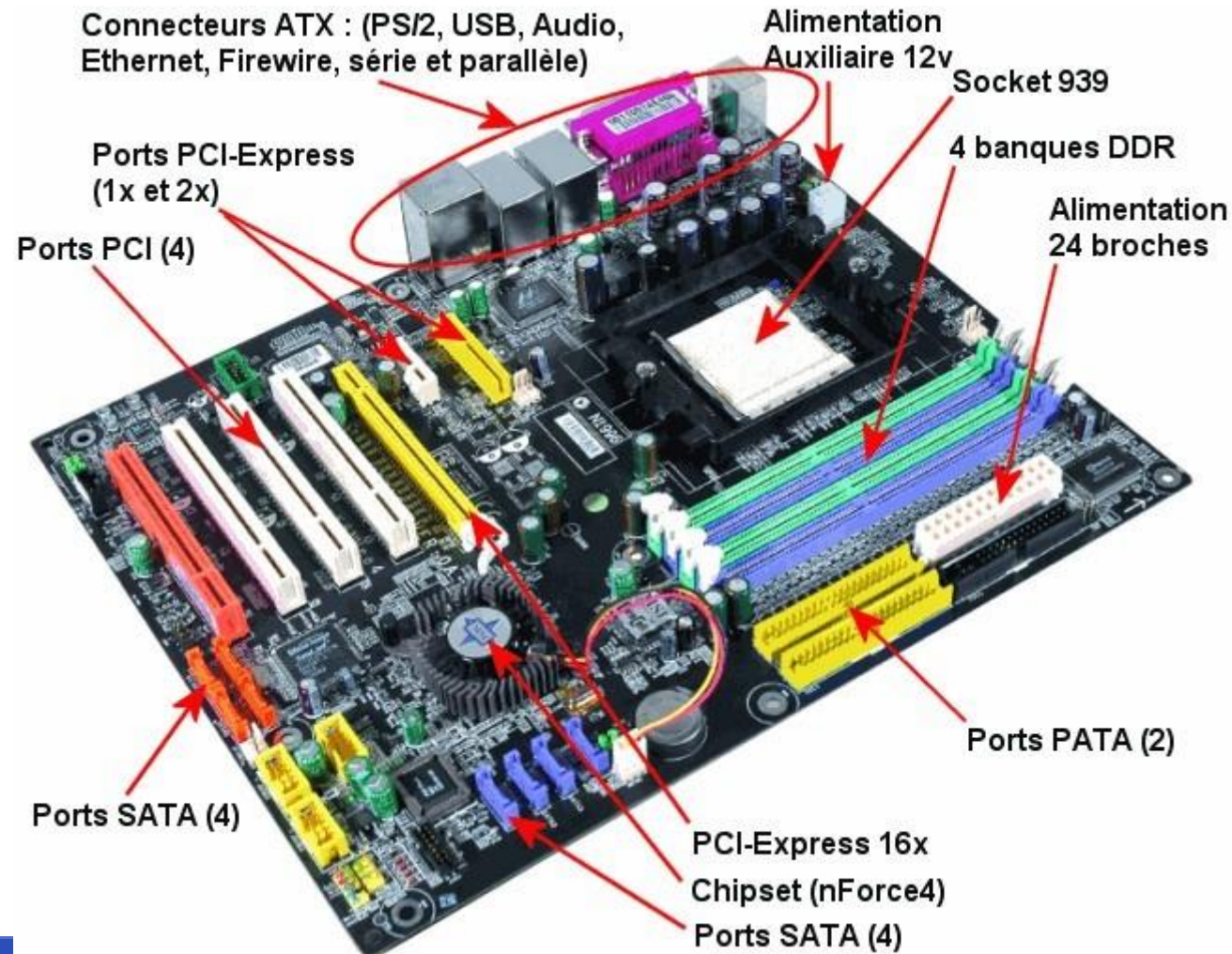
Les composants de l'ordinateur

La carte mère



Les composants de l'ordinateur

La carte mère



Les périphériques

Les périphériques

- Un ordinateur se compose d'une unité centrale et des périphériques.
- Tous les périphériques sont branchés sur l'unité centrale

Les périphériques



Les périphériques



Les périphériques

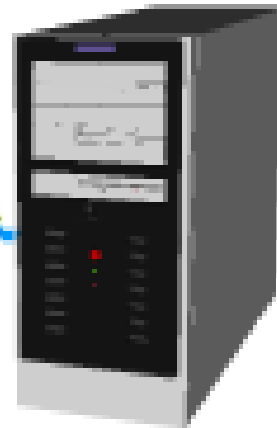
Il existe trois types de périphériques

- Périphériques d'entrée
- Périphériques de sortie
- Périphérique d'entrée et sortie

Les périphériques

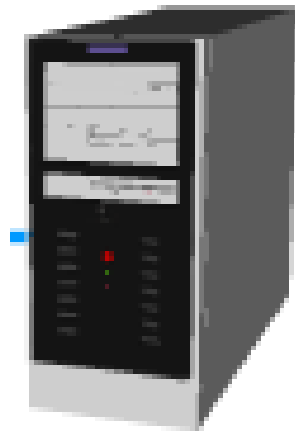
Les périphériques d'entrée : Ils envoient des informations à l'unité centrale

Périphérique
d'entrée



Les périphériques

Les périphériques de sortie: Ils reçoivent des informations de l'unité centrale



Périphérique
de sortie

Les périphériques

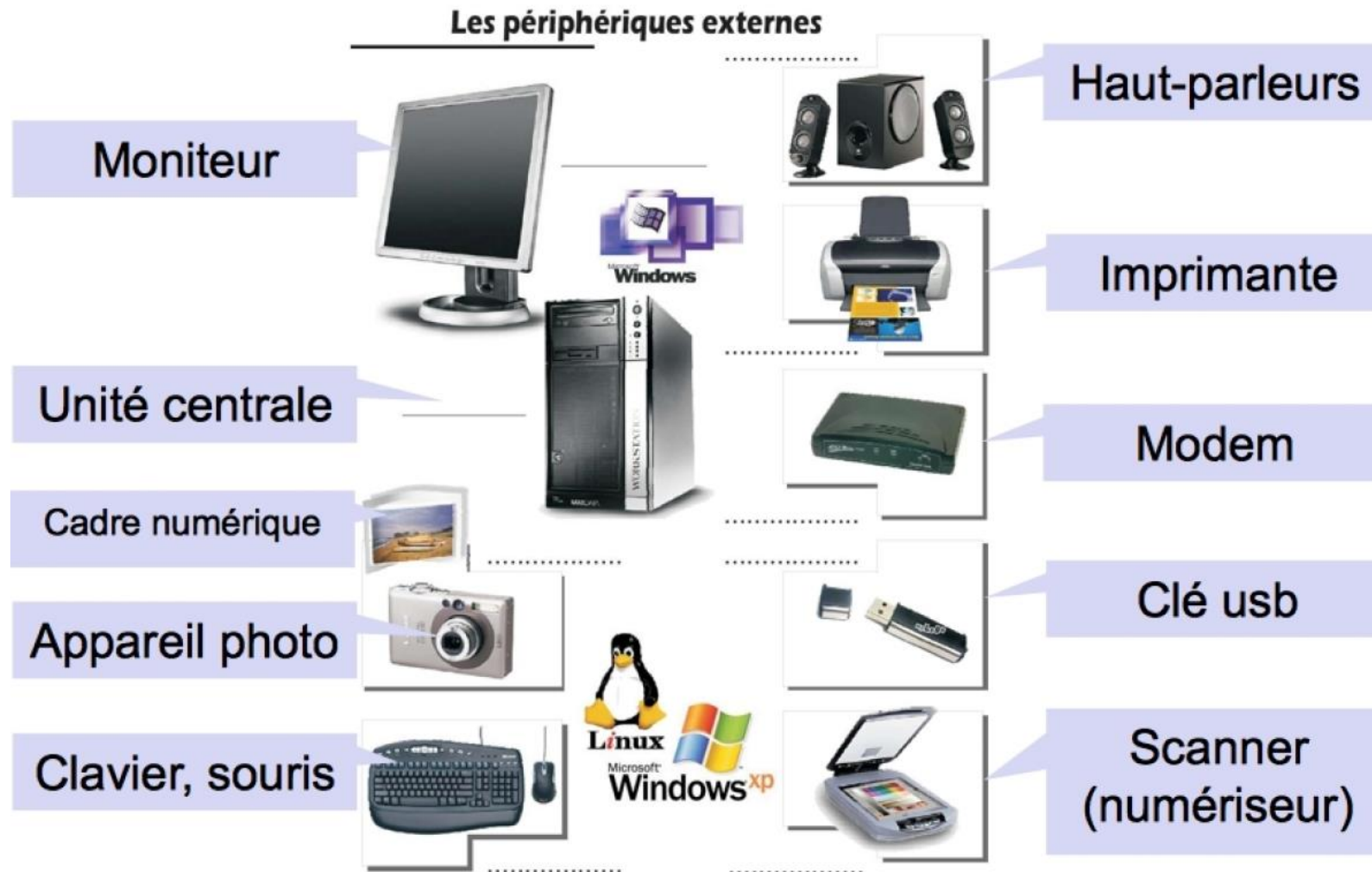
Les périphériques d'entrée et sortie: Ils envoient des informations à l'unité centrale et en reçoivent

Périphérique
d'entrée et
sortie

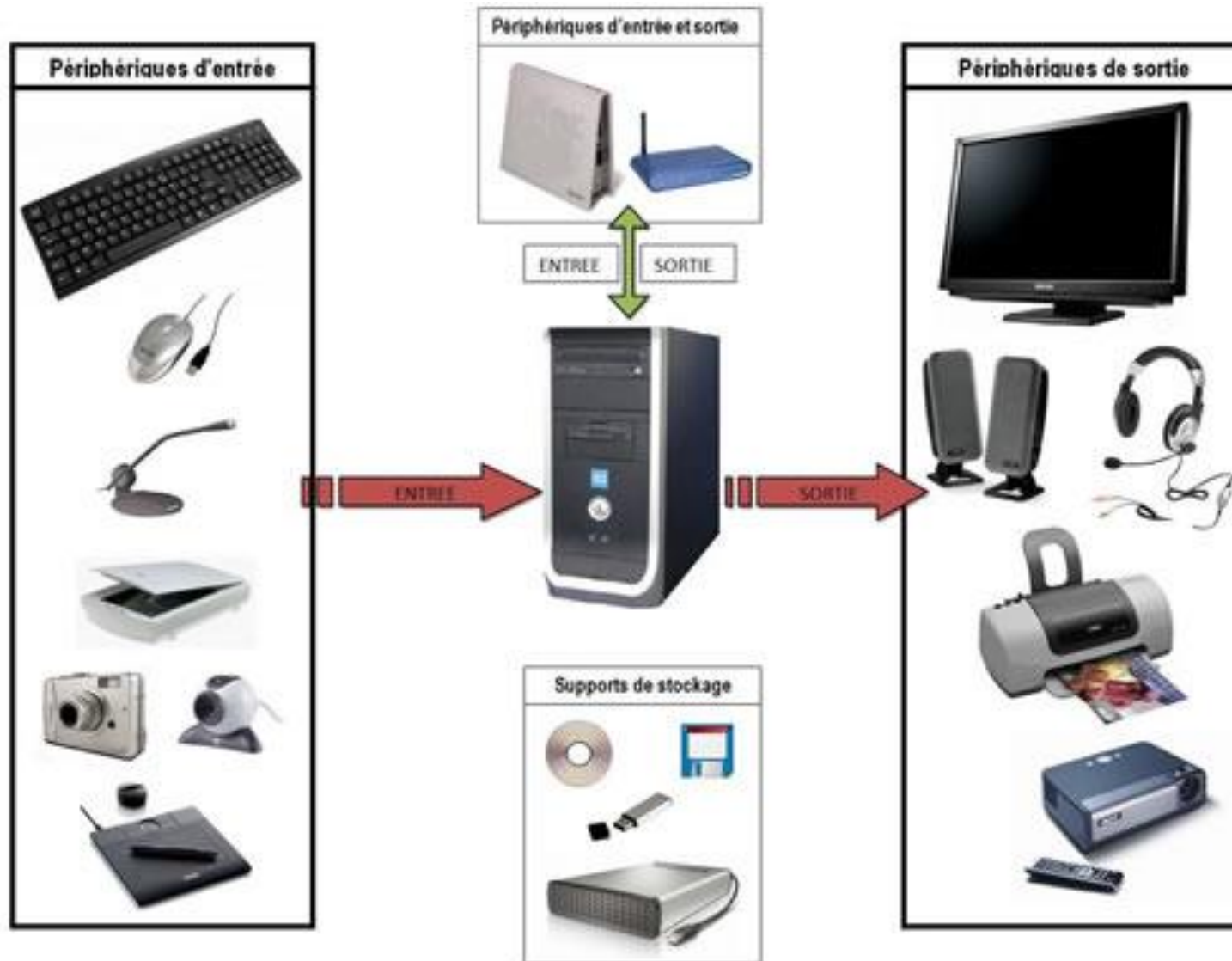


Exemple de périphériques

Les périphériques



Les périphériques



Les systèmes d'exploitation

Système d'exploitation

Le SE soustrait le matériel au regard du programmeur et offre une présentation agréable des fichiers. Un SE a ainsi deux objectifs principaux :

- **présentation** : Il propose à l'utilisateur une abstraction plus simple et plus agréable que le matériel : une machine virtuelle.
- **gestion** : il ordonne et contrôle l'allocation des processeurs, des mémoires, des icônes et fenêtres, des périphériques, des réseaux entre les programmes qui les utilisent. Il assiste les programmes utilisateurs. Il protège les utilisateurs dans le cas d'usage partagé

Éléments de base d'un système d'exploitation

Les principales fonctions assurées par un SE sont les suivantes :

- gestion de la mémoire principale et des mémoires secondaires,
- exécution des E/S à faible débit (terminaux, imprimantes) ou haut débit (disques, bandes),
- multiprogrammation, temps partagé, parallélisme : interruption, ordonnancement, répartition en mémoire, partage des données

Éléments de base d'un système d'exploitation

- lancement des outils du système (compilateurs, environnement utilisateur,...) et des outils pour l'administrateur du système (création de points d'entrée, modification de privilèges,...),
- lancement des travaux,
- protection, sécurité ; facturation des services,
- réseaux

Éléments de base d'un système d'exploitation

L'interface entre un SE et les programmes utilisateurs est constituée d'un ensemble d'instructions étendues, spécifiques d'un SE, ou appels système. Généralement, les appels système concernent soit les processus, soit le système de gestion de fichiers (SGF).

Les processus

Un processus est un programme qui s'exécute, ainsi que ses données, sa pile, son compteur ordinal, son pointeur de pile et les autres contenus de registres nécessaires à son exécution.

Les interruptions

- Une interruption est une commutation du mot d'état provoquée par un signal généré par le matériel.
- Ce signal est la conséquence d'un événement interne au processus, résultant de son exécution, ou bien extérieur et indépendant de son exécution.
- Le signal va modifier la valeur d'un indicateur qui est consulté par le SE. Celui-ci est ainsi informé de l'arrivée de l'interruption et de son origine.

Les interruptions

On distingue au moins 3 niveaux d'interruption :

- les interruptions externes : panne, intervention de l'opérateur,
- les déroutements qui proviennent d'une situation exceptionnelle ou d'une erreur liée à l'instruction en cours d'exécution (division par 0, débordement, ...)
- les appels système

Les ressources

- On appelle ressource tout ce qui est nécessaire à l'avancement d'un processus (continuation ou progression de l'exécution) : processeur, mémoire, périphérique, bus, réseau, compilateur, fichier, message d'un autre processus, etc.
- Un défaut de ressource peut provoquer la mise en attente d'un processus.

Les ressources

- Un processus demande au SE l'accès à une ressource. Certaines demandes sont implicites ou permanentes (la ressource processeur).
- Le SE alloue une ressource à un processus.
- Une fois une ressource allouée, le processus a le droit de l'utiliser jusqu'à ce qu'il libère la ressource ou jusqu'à ce que le SE reprenne la ressource (on parle en ce cas de ressource préemptible, de préemption).

Les ressources

On dit qu'une ressource est en mode d'accès exclusif si elle ne peut être allouée à plus d'un processus à la fois. Sinon, on parle de mode d'accès partagé. Un processus possédant une ressource peut dans certains cas en modifier le mode d'accès.

L'ordonnancement

On appelle ordonnancement la stratégie d'attribution des ressources aux processus qui en font la demande.

Différents critères peuvent être pris en compte :

- temps moyen d'exécution minimal
- temps de réponse borné pour les systèmes interactifs
- taux d'utilisation élevé de l'UC
- respect de la date d'exécution au plus tard, pour le temps réel, etc.

Le système de gestion de fichiers

Une des fonctions d'un SE est de masquer les spécificités des disques et des autres périphériques d'E/S et d'offrir au programmeur un modèle de manipulation des fichiers agréable et indépendant du matériel utilisé.

Le système de gestion de fichiers

- Les appels système permettent de créer des fichiers, de les supprimer, de lire et d'écrire dans un fichier.
- Il faut également ouvrir un fichier avant de l'utiliser, le fermer ultérieurement.
- Les fichiers sont regroupés en répertoires arborescents; ils sont accessibles en énonçant leur chemin d'accès (chemin d'accès absolu à partir de la racine ou bien chemin d'accès relatif dans le cadre du répertoire de travail courant). Le SE gère également la protection des fichiers.

Exemple de système d'exploitations

Les systèmes d'exploitation



Chapitre 3:

Architecture des ensembles d'instructions

Plan

1. Introduction
2. Le processeur
3. Interface matériel-logiciel
4. Classification des architectures: RISC vs CISC
5. Jeux d'instructions
6. Les registres
7. Mode d'adressage
8. Langage machine
9. Les pipelines

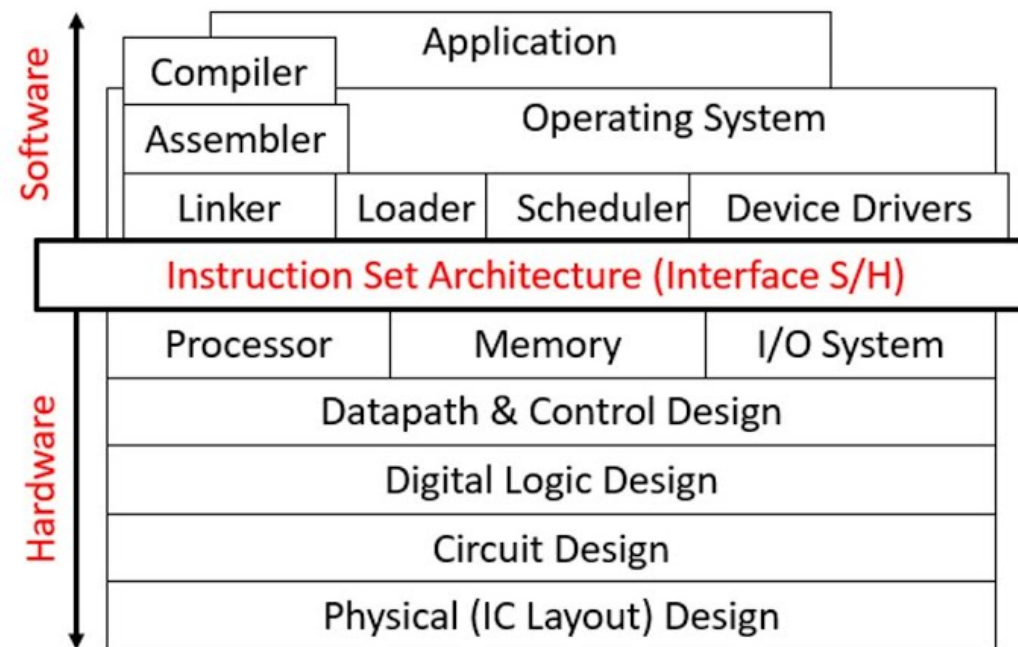
Introduction

Introduction

- L'architecture des ensembles d'instructions ou ISA (Instruction Set Architecture) est une interface entre le matériel (processeur) et le logiciel.
- Elle définit l'ensemble des instructions que le processeur peut exécuter, leur format, et la manière dont elles interagissent avec la mémoire et les périphériques.

Introduction

| Instruction Set Architecture (ISA)



Introduction

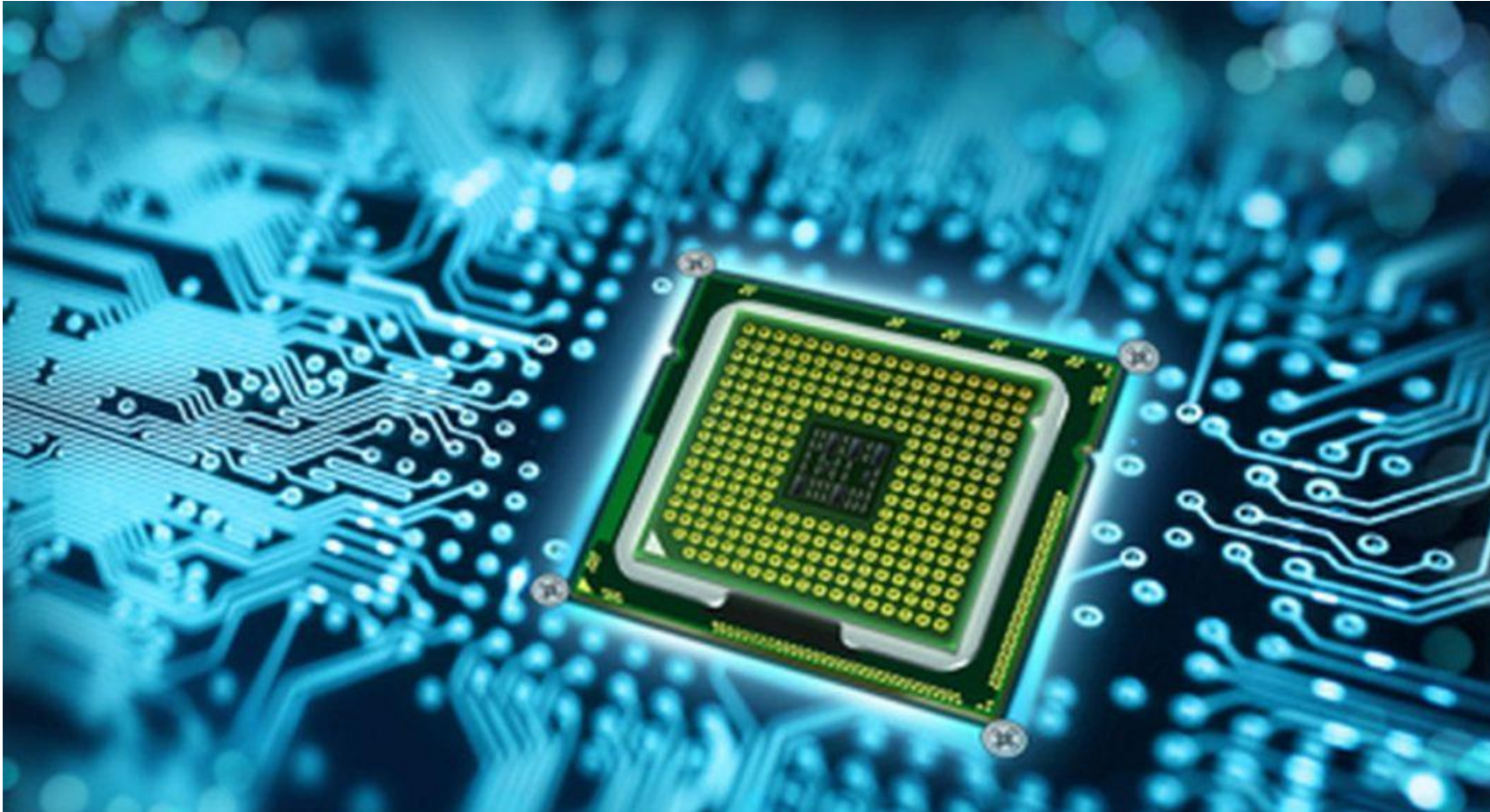
- Influence la conception du matériel et du logiciel
- Affecte la performance, la consommation énergétique et la compatibilité

Le Processeur

Le processeur

- Le processeur est la partie centrale de l'ordinateur, souvent qualifié comme le cerveau responsable d'exécuter les instructions.
- Il fonctionne comme une unité de traitement qui interprète et exécute les commandes fournies par les programmes.

Le processeur



Les composants principaux (Processeur)

Le processeur se compose principalement de:

- Unité arithmétique et logique (UAL)
- Unité de contrôle
- Les registres

Unité Arithmétique et Logique (UAL)

Le processeur se compose principalement de:
Effectuer les opérations mathématiques (addition, soustraction, multiplication, division) et logiques (AND, OR, NOT, XOR).

Unité de contrôle

- Coordonne l'ensemble des activités du processeur.
- Décodage des instructions pour déterminer les actions à effectuer.
- Gestion des signaux de contrôle pour synchroniser les différents composants.

Les registres

- Stocker temporairement des données, des adresses ou des résultats intermédiaires.
- Faciliter un accès rapide aux données par le processeur.

Les types de registres

- Registres de données (stockent des valeurs à utiliser immédiatement).
- Registres d'instructions (contiennent les instructions à exécuter).
- Compteur ordinal ou Program Counter (PC) : Indique l'adresse de la prochaine instruction à exécuter.

Les composants principaux

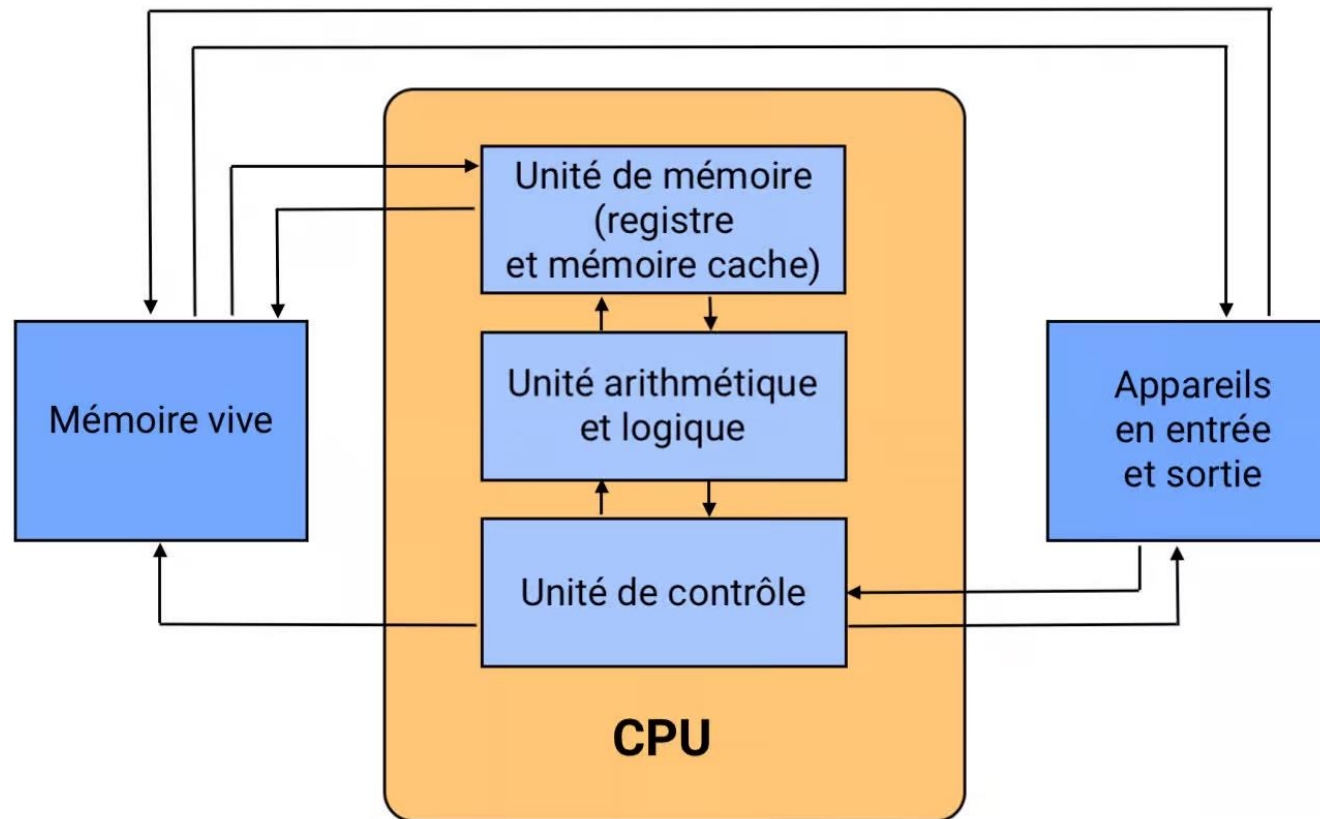


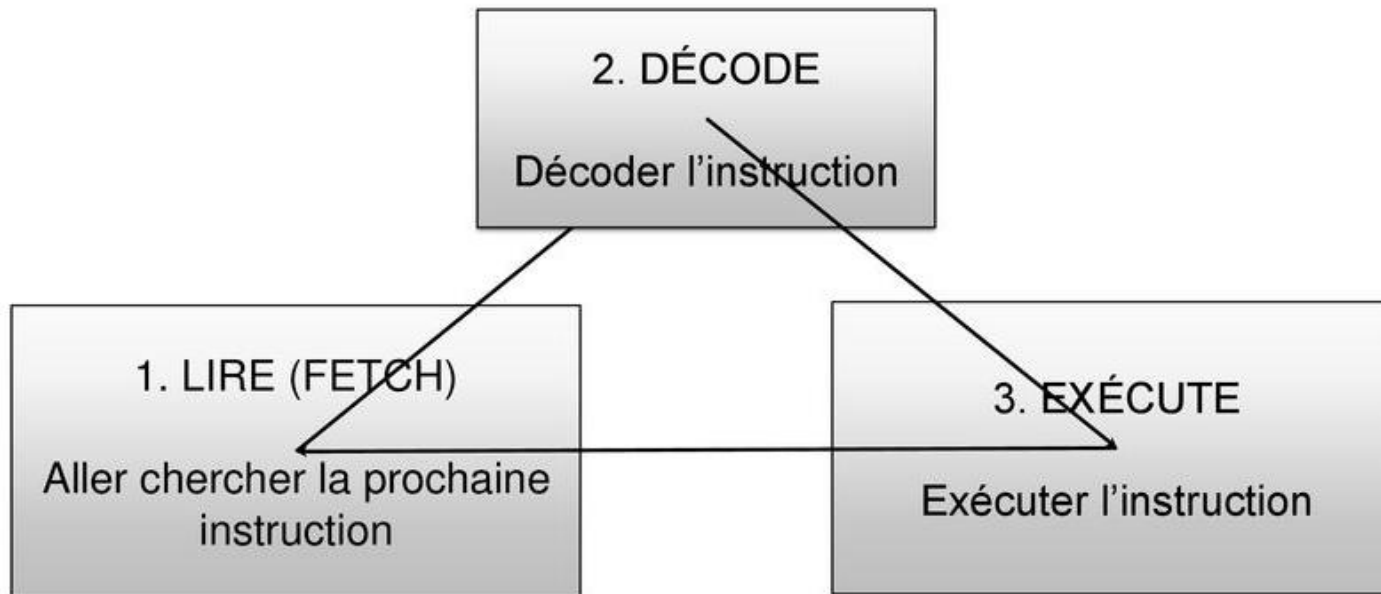
Schéma de fonctionnement d'un processeur

Cycle d'instructions

Le cycle d'instructions décrit les étapes qu'un processeur suit pour exécuter une commande. Il comporte généralement trois étapes principales :

- Fetch (lecture de l'instruction)
- Decode (Décodage de l'instruction)
- Execute (Exécution de l'instruction)

Cycle d'instructions



Cycle d'instructions

Ce fonctionnement cyclique permet au processeur de traiter des milliards d'instructions par seconde dans les systèmes modernes grâce aux pipelines qui permettent de chevaucher les instructions.

Métrique pour mesurer la performance d'un processeur:

Calcul du CPI (Cycle Per Instruction)

- Le CPI (Cycle Per Instruction) est une métrique clé pour évaluer les performances d'un processeur.
- Il représente le nombre moyen de cycles d'horloge nécessaires pour exécuter une instruction.
- Autrement, pour exécuter une instruction, on aura besoin de combien de cycles.

Calcul de CPI (Cycle Per Instruction)

Formule de base

$$CPI = \frac{\sum (\text{Nombre d'instructions par type} \times CPI \text{ par type})}{\text{Nombre total d'instructions}}$$

CPI par type : Nombre moyen de cycles requis pour chaque type d'instruction.

Calcul de CPI (Cycle Per Instruction)

Exemple de calcul

Supposons qu'un programme exécute les types d'instructions suivants :

- 100 instructions arithmétiques, chaque instruction nécessitant 2 cycles.
- 50 instructions de mémoire, chaque instruction nécessitant 4 cycles.
- 150 instructions de branchement, chaque instruction nécessitant 3 cycles.

Calcul de CPI (Cycle Per Instruction)

Étape 1 : Calcul du nombre total de cycles par type

- Arithmétiques : $100 \times 2 = 200$ cycles.
- Mémoire : $50 \times 4 = 200$ cycles.
- Branchements : $150 \times 3 = 450$ cycles.

Calcul de CPI (Cycle Per Instruction)

Étape 2 : Total des cycles d'horloge et instructions

- Total des cycles : $200+200+450=850$.
- Total des instructions : $100+50+150=300$.

Étape 3 : Calcul du CPI moyen

$$CPI = \frac{850}{300} \approx 2.83$$

Interprétation du CPI

Le CPI (Cycle Per Instruction) permet d'évaluer l'efficacité d'un processeur. Selon sa valeur, on peut déduire plusieurs éléments :

- Faible CPI : Indique un processeur rapide et optimisé pour exécuter les instructions en peu de cycle
- Élevé CPI : Signale des instructions complexes nécessitant plus de cycles pour leur exécution

Fourchette à suivre pour CPI:

Si < 1 processeur très performant
= 1 processeur normal et optimisé
 ≥ 3 n'est pas performant

Calcul du MIPS (Million Instructions Per Second)

Le MIPS mesure le nombre d'instructions exécutées par un processeur en une seconde. C'est une autre métrique utilisée pour évaluer les performances.

Formule du MIPS

$$MIPS = \frac{\text{Fréquence du processeur (Hz)}}{CPI * 10^6}$$

Calcul du MIPS (Million Instructions Per Second)

Exemple de calcul

Supposons que :

La fréquence du processeur est 3 GHz (3×10^9 Hz).

Le CPI est 2.5.

Calcul du MIPS

$$MIPS = \frac{3 * 10^9}{2.5 * 10^6} = 1200 MIPS$$

Cela signifie que le processeur peut exécuter 1200 millions d'instructions par seconde.

Calcul du MIPS (Million Instructions Per Second)

Exercice d'application

Un processeur fonctionne à une fréquence de 2.5 GHz. Un programme donné exécute plusieurs types d'instructions, avec les détails suivants :

1. Le programme contient 200 millions d'instructions.
2. Les instructions se répartissent comme suit :
60 millions d'instructions arithmétiques (2 cycles par instruction).
100 millions d'instructions mémoire (4 cycles par instruction).
40 millions d'instructions de branchement (3 cycles par instruction).

Questions :

1. Calculez le nombre total de cycles d'horloge nécessaires pour exécuter ce programme.
2. Déterminez le CPI moyen pour ce programme.
3. Calculez les MIPS pour ce programme sur ce processeur.
4. Si le programme est optimisé et nécessite désormais 300 millions de cycles pour s'exécuter complètement, quel est le nouveau CPI ?

Interface matériel-logiciel

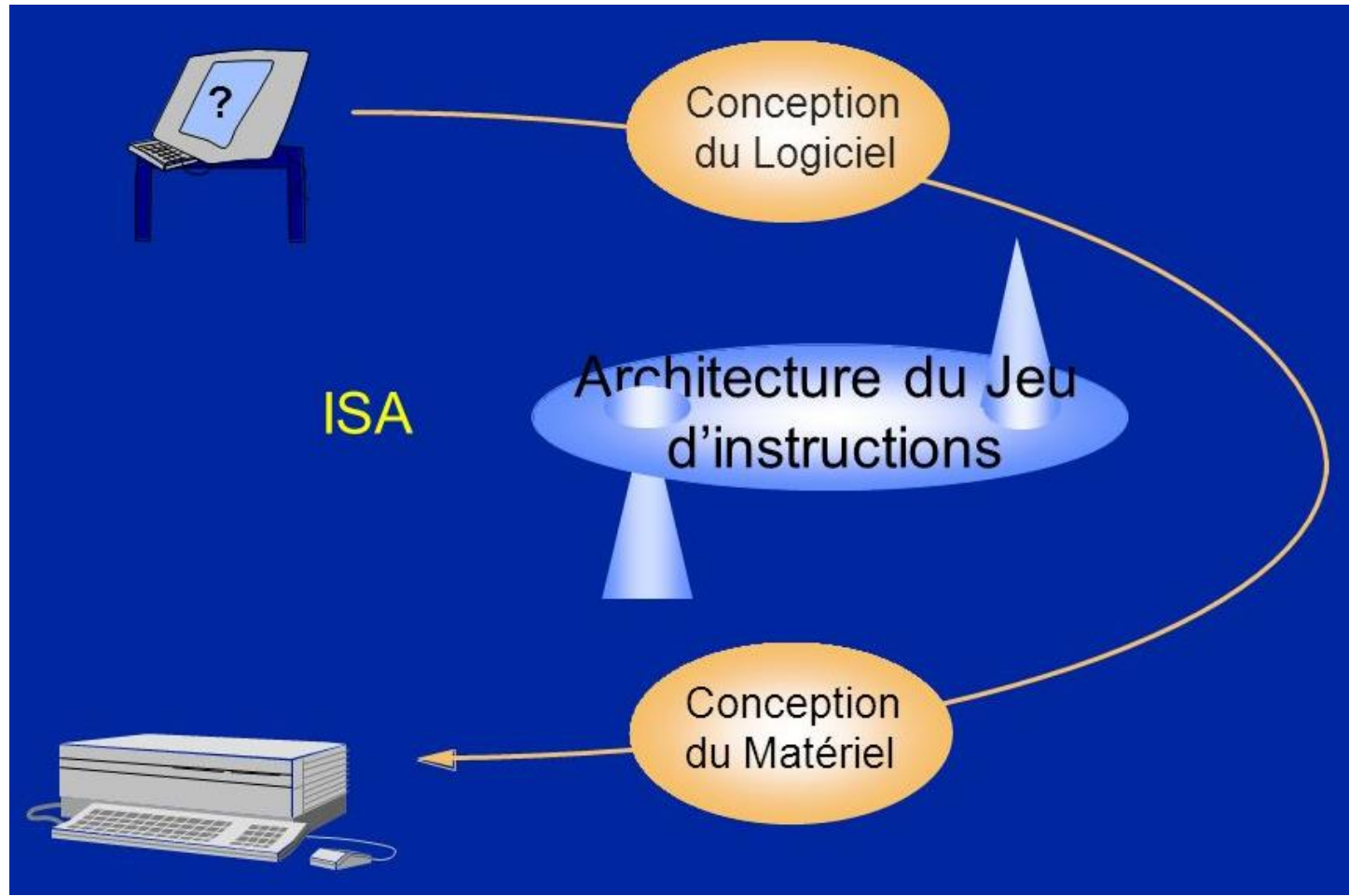
Interface matériel-logiciel

L'interface entre le matériel et le logiciel permet au processeur d'exécuter des commandes écrites en langage de haut niveau.

Interface matériel-logiciel

- Traduction des instructions (compilateurs, assembleurs).
- Microprogrammes dans certains processeurs pour interpréter les instructions complexes.

Interface matériel-logiciel



Classification des architectures : RISC vs CISC

Classification des architectures

- L'architecture des ensembles d'instructions se divise principalement en deux grandes catégories :
 - CISC (Complex Instruction Set Computing)
 - RISC (Reduced Instruction Set Computing).
- Ces deux approches reflètent des philosophies différentes dans la conception des processeurs.
- Il s'agit de deux types d'architectures de microprocesseurs qui diffèrent dans la façon dont elles exécutent les instructions

CISC (Complex Instruction Set Computing)

Les processeurs CISC disposent d'un ensemble d'instructions plus important et plus complexe qui peuvent effectuer plusieurs opérations en une seule instruction, mais dont l'exécution peut prendre plus de cycles d'horloge.

CISC (Complex Instruction Set Computing)

Autrement dit, chaque instruction peut effectuer plusieurs opérations en une seule commande, comme charger des données depuis la mémoire, les manipuler et les sauvegarder.

RISC (Reduced Instruction Set Computing)

Les processeurs RISC ont un ensemble d'instructions plus petit et plus simple qui peut être exécuté en un ou plusieurs cycles d'horloge.

RISC (Reduced Instruction Set Computing)

Autrement dit, les architectures RISC utilisent un ensemble d'instructions réduit, chaque instruction étant simple et rapide à exécuter.

L'objectif est de maximiser la vitesse d'exécution des instructions en utilisant un matériel optimisé.

Comparaison entre CISC et RISC

Caractéristiques	CISC	RISC
Complexité des instructions	Complexes et polyvalentes	Simplees et uniformes
Exécution des instructions	Plusieurs cycles d'horloge	Un cycle par instruction
Taille du matériel	Complexe et volumineux	Simple et optimisé
Consommation énergétique	Plus élevée	Plus faible
Pipeline	Moins efficace	Très efficace
Exemples	x86, Intel, AMD	ARM, RISC-V, MIPS

Jeux d'instructions

Jeux d'instructions

Un jeu d'instructions est l'ensemble des commandes ou instructions qu'un processeur peut comprendre et exécuter. Chaque instruction est composée de divers éléments permettant de spécifier l'opération à effectuer et les données à manipuler.

Classification des instructions

Les jeux d'instructions peuvent être classifiés en plusieurs catégories selon leur nature et leur fonctionnalité :

1. Instructions arithmétiques

Effectuent des opérations mathématiques de base :

Addition : ex : (ADD R1, R2).

Soustraction : (ex : SUB R1, R2).

Multiplication : (ex : MUL R1, R2).

Classification des instructions

2. Instructions logiques

Réalisent des opérations logiques bit à bit :

AND

OR

NOT

XOR

Classification des instructions

3. Instructions de transfert de données

Permettent de déplacer ou copier des données :

Chargement : Transfère une donnée de la mémoire vers un registre (ex : LOAD R1, ADDRESS).

Stockage : Écrit une donnée d'un registre dans la mémoire (ex : STORE R1, ADDRESS).

Déplacement : Copie une donnée d'un registre vers un autre (ex : MOVE R1, R2).

Classification des instructions

4. Instructions de contrôle

Modifient le flux d'exécution du programme :

- **Sauts** : Changement conditionnel ou inconditionnel de l'adresse d'exécution (ex : JUMP LABEL).
- **Appels** de fonctions : Permettent de naviguer vers une sous-routine (ex : CALL FUNCTION).
- **Retour** : Reprend l'exécution après une sous-routine (ex : RETURN).

Structure d'une instruction

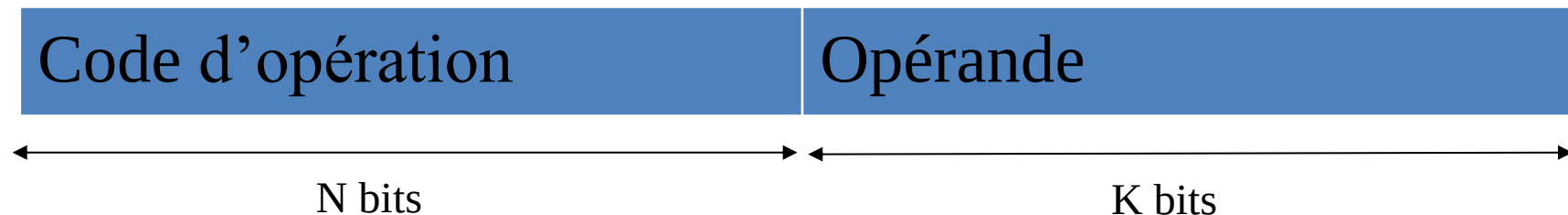
Une instruction peut être structurée comme suit:

- Opcode (Operation Code): spécifie l'opération à exécuter (ADD, SUB, LOAD, ...)
- Opérandes: Ce sont les données ou les références de données impliquées dans l'instruction.
- Mode d'adressage: Détermine comment les opérandes sont interprétés (ou comment on accède aux opérandes).

Structure d'une instruction

L'instruction est découpée en deux parties :

- Code opération (code instruction) : un code sur N bits qui indique quelle instruction.
- Le champ opérande : qui contient la donnée ou la référence (adresse) à la donnée.



Structure d'une instruction

Le format d'une instruction peut ne pas être le même pour toutes les instructions.

Le champ opérande peut être découpé à son tour en *plusieurs champs*

Machine à 3 adresses

Dans ce type de machine, pour chaque instruction il faut préciser :

- l'adresse du premier opérande
- du deuxième opérande
- et l'emplacement du résultat

Code d'opération	Opérande1	Opérande2	Opérande3
------------------	-----------	-----------	-----------

Exemple:

ADD A,B,C ($C \leftarrow A+B$)

Machine à 2 adresses

Dans ce type de machine pour chaque instruction il faut préciser :

- l'adresse du premier opérande
 - du deuxième opérande ,
- l'adresse de résultat est implicitement l'adresse du deuxième opérande.

Code d'opération	Opérande1	Opérande2
------------------	-----------	-----------

Exemple:

ADD A,B ($B \leftarrow A+B$)

Machine à 1 adresse

Dans ce type de machine pour chaque instruction il faut préciser uniquement l'adresse du deuxième opérande.

- Le premier opérande existe dans le registre accumulateur.
- Le résultat est mis dans le registre accumulateur

Code d'opération	Opérande2
------------------	-----------

Exemple:

ADD A

$(ACC \leftarrow (ACC) + A)$

Ce type de machine est le plus utilisé.

Mode d'adressage

Mode d'adressage (comment on accède aux opérandes)

- Le champ opérande contient la donnée ou la référence (adresse) à la donnée.
- Le mode d'adressage définit la manière dont le microprocesseur va accéder à l'opérande.
- Le code opération de l'instruction comporte un ensemble de bits pour indiquer le mode d'adressage.
- Les modes d'adressage les plus utilisés sont : ***Immédiat, Direct, Indirect, Indexé, relatif***

Adressage immédiat

L'opérande existe dans le champ adresse de l'instruction

Code d'opération	Opérande
------------------	----------

Exemple:

ADD 150

ADD	150
-----	-----

Cette commande va avoir l'effet suivant:

$ACC \leftarrow (ACC) + 150$

Si le registre accumulateur contient la valeur 200 alors après exécution son contenu sera à 350.

Adressage direct

Le champ opérande de l'instruction contient l'adresse de l'opérande (emplacement en mémoire)

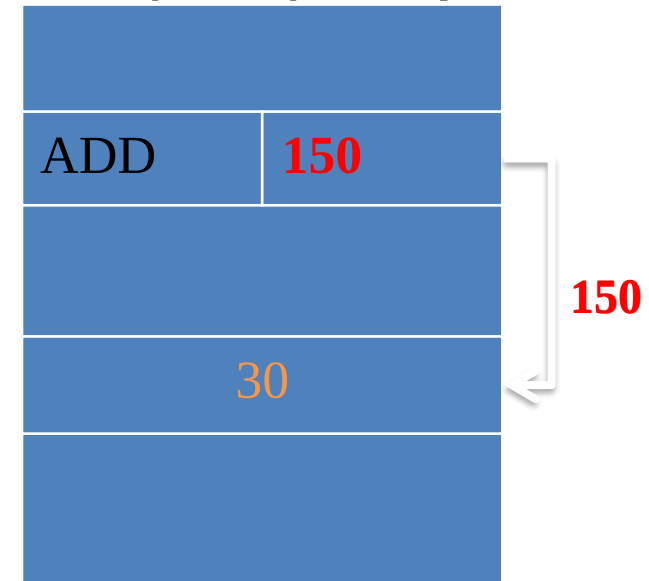
- Pour réaliser l'opération, il faut le récupérer (lire) l'opérande à partir de la mémoire.

$ACC \leftarrow (ACC) + (ADR)$

Exemple:

On suppose que l'accumulateur contient une valeur 20

A la fin de l'exécution nous allons avoir la valeur 50 i.e (20 + 30)



Adressage indirect

Le champ adresse contient l'adresse de l'opérande.

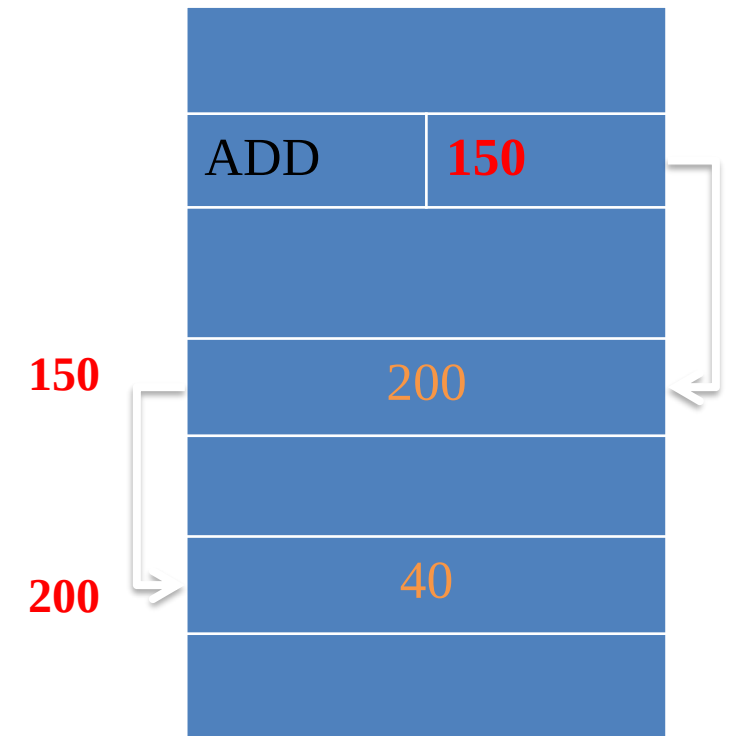
Pour réaliser l'opération il faut :

- Récupérer l'adresse de l'opérande à partir de la mémoire.
- Par la suite, il faut chercher l'opérande à partir de la mémoire.

$ACC \leftarrow (ACC) + ((ADR))$

Exemple :

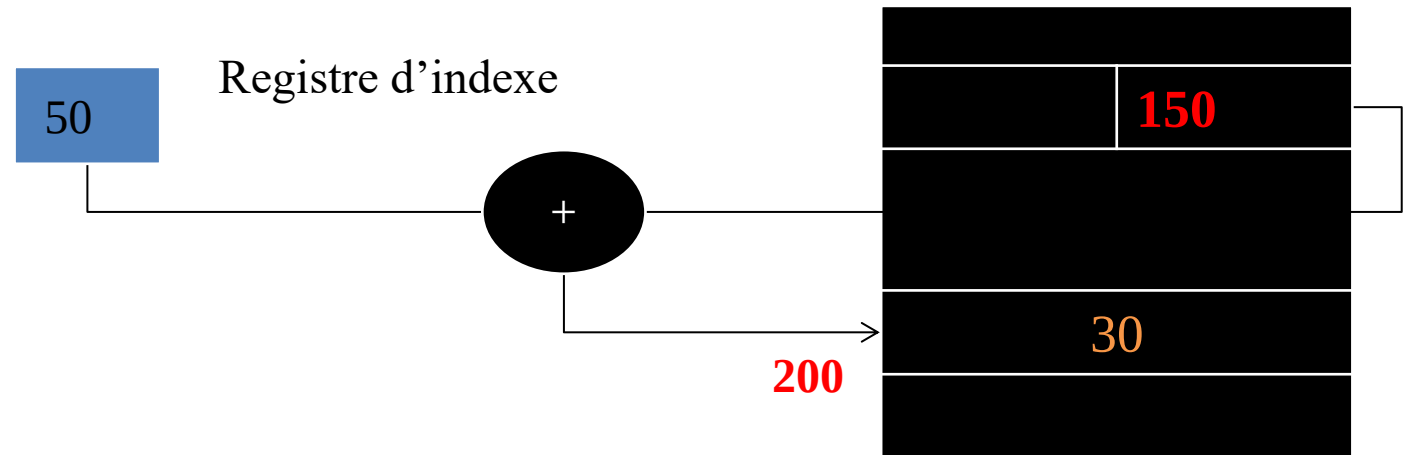
- Initialement l'accumulateur contient la valeur 20
- Il faut récupérer l'adresse de l'adresse (150).
- Récupérer l'adresse de l'opérande à partir de l'adresse 150 (la valeur 200)
- Récupérer la valeur de l'opérande à partir de l'adresse 200 (la valeur 40)
- Additionner la valeur 40 avec le contenu de l'accumulateur (20) et nous allons avoir la valeur 60



Adressage indexé

L'adresse effective de l'opérande est relatif à une zone mémoire.

- L'adresse de cette zone se trouve dans un registre spécial (registre indexe).
- Adresse opérande = $ADR + (X)$

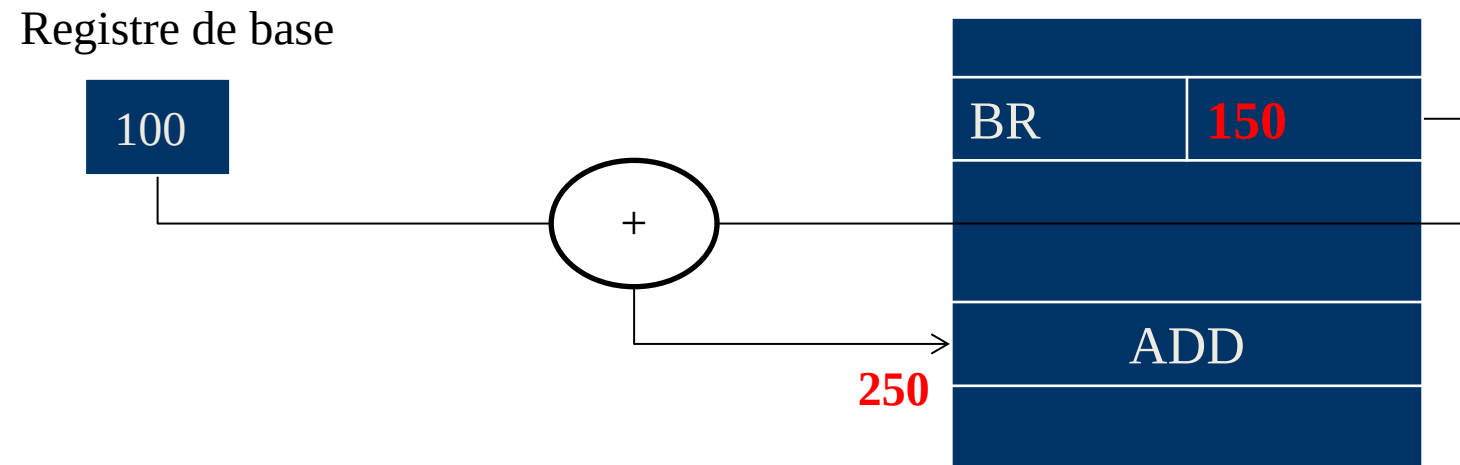


Remarque : si ADR ne contient pas une valeur immédiate
alors Adresse opérande = $(ADR) + (X)$

Adressage relatif

- L'adresse effective de l'opérande est relatif a une zone mémoire.
- L'adresse de cette zone se trouve dans un registre spécial (registre de base).
- Ce mode d'adressage est utilisée pour les instructions de branchement.

Adresse = ADR + (base)



Les registres

Les registres

Les registres sont utilisés pour stocker temporairement des données, des résultats intermédiaires, ou des valeurs nécessaires aux calculs.

Exemple : Dans une instruction comme `ADD R1, R2, R3`, les registres généraux R1, R2, et R3 contiennent les opérandes et le résultat.

Les registres spécialisés

Ces registres remplissent des fonctions spécifiques liées au contrôle et à la gestion des opérations dans le processeur

Les registres spécialisés /Compteur Ordinal (Program Counter - PC)

Indique l'adresse de la prochaine instruction à exécuter. Incrémenté automatiquement après chaque instruction ou modifié lors d'un saut.

Exemple : Après une instruction JUMP 200, le PC contient 200

Les registres spécialisés

/Registre d'état

Les registres d'état contiennent des indicateurs (ou flags) décrivant les résultats des opérations :

- ***Zero flag*** : Indique que le résultat est nul.
- ***Carry flag*** : Signale un dépassement lors d'une addition ou soustraction.
- ***Overflow flag*** : Détecte une erreur due à une limite d'encodage (par exemple, dépassement de bit de signe).

Les registres spécialisés

/Registre d'adresse

Ils sont utilisés pour stocker des adresses mémoire.

Les types courants :

- ***Base Register*** : Contient une adresse de base pour un segment mémoire.
- ***Index Register*** : Contient un décalage ou une adresse relative.

Exemple : Dans un mode d'adressage indexé, LOAD R1, BASE+INDEX.

Le rôle des registres

- Accès rapide aux données
- Réduction des accès à la mémoire principale
- Optimisation des performances

Les registres

Exemple pratique

Prenons une opération de multiplication impliquant un registre spécialisé et des registres généraux :

1. Instruction : MUL R3, R1, R2

Rôle des registres :

R1 : Contient le premier opérande.

R2 : Contient le second opérande.

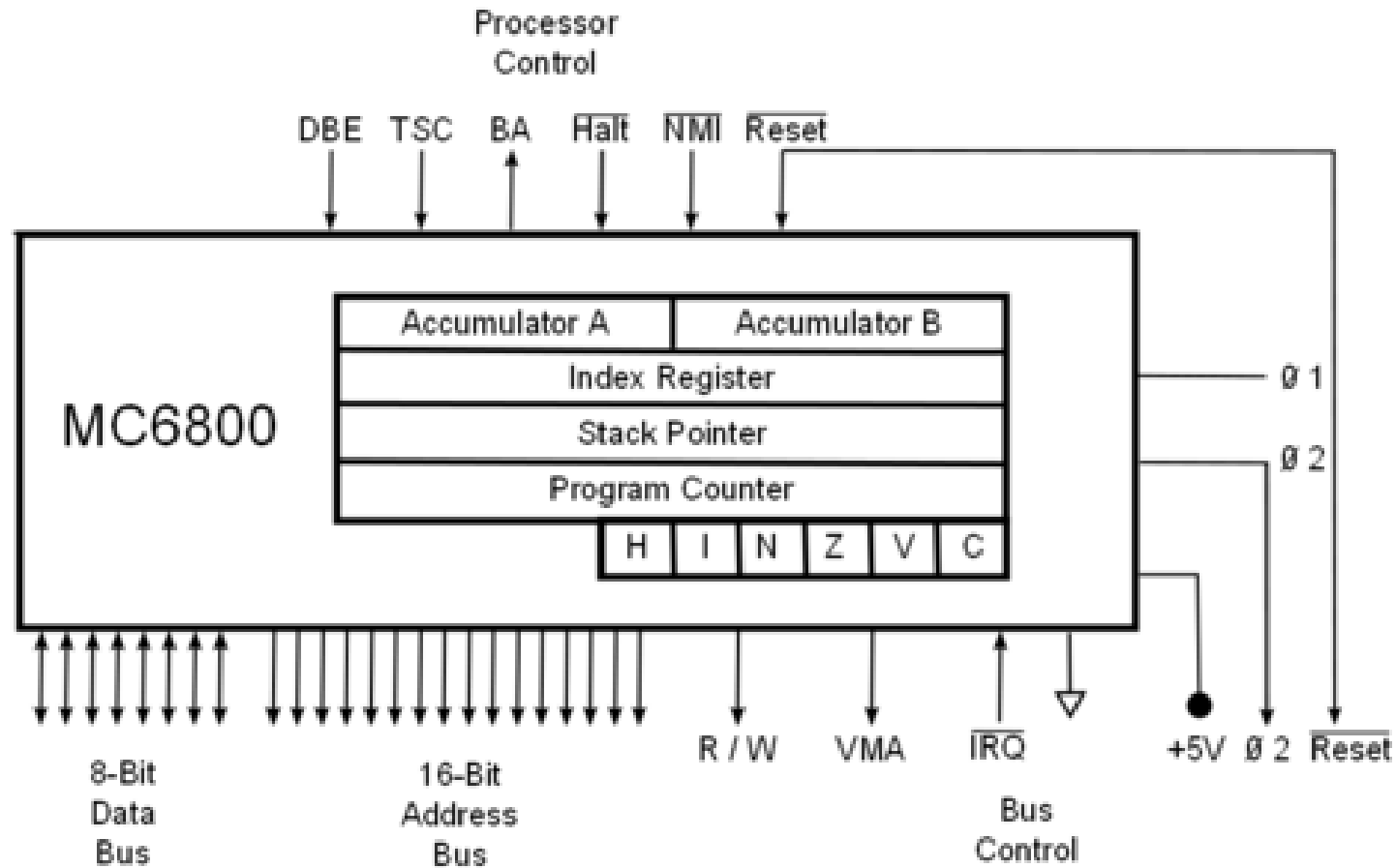
R3 : Stocke le résultat de la multiplication.

2. Registre d'état (Status Register) :

Si le résultat est nul, le Zero flag sera activé.

En cas de dépassement, le Carry flag sera activé.

Les registres



Les registres d'un processeur Motorola 6800

Pipeline

Pipeline

Un pipeline est une technique utilisée dans les processeurs pour améliorer leur performance. L'idée est de diviser le cycle d'instruction en plusieurs étapes indépendantes et de les exécuter simultanément pour différentes instructions. Cela permet au processeur d'atteindre une exécution plus rapide et plus efficace.

Pipeline

L'idée générale est lancer le traitement d'une instruction avant que la précédente ne soit terminée.

Pipeline

Un pipeline est une technique utilisée dans les processeurs pour améliorer leur performance. L'idée est de diviser le cycle d'instruction en plusieurs étapes indépendantes et de les exécuter simultanément pour différentes instructions. Cela permet au processeur d'atteindre une exécution plus rapide et plus efficace.

Pipeline

Exemple simplifié

Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode	Fetch
4	Write-back	Execute	Decode
5		Write-back	Execute
6			Write-back

Problème des pipelines

Instructions :

1. Instruction 1 : Ajouter R1 et R2 $\rightarrow$ stocker dans R3. (ADD R3, R1, R2)
2. Instruction 2 : Multiplier R3 par R4 $\rightarrow$ stocker dans R5. (MUL R5, R3, R4)
3. Instruction 3 : Ajouter R5 et R6 $\rightarrow$ stocker dans R7. (ADD R7, R5, R6)

Problème des pipelines

Pipeline Étapes :

Cycle	Instruction 1	Instruction 2	Instruction 3
1	Fetch		
2	Decode	Fetch	
3	Execute	Decode (Bloqué, attend R3)	Fetch
4	Write-back	Execute	Decode (bloqué, attend R5)
5		Write-back	Execute
6			Write-back

Problème des pipelines

Solution Possible :

- ***Forwarding*** : Transférer directement le résultat de R3 ou R5 dès qu'il est produit (à l'étape Execute), au lieu d'attendre la fin du Write-back.
- ***Stall*** (comme dans cet exemple) : Insérer des cycles d'attente pour résoudre les conflits.

Pipeline Étapes avec Cycles d'Attente :

Cycle	Instruction 1	Instruction 2	Instruction 3	Notes
1	Fetch			Lecture de l'instruction 1
2	Decode	Fetch		Lecture de l'instruction 2
3	Execute	Decode (Bloqué, attend R3)		Instruction 2 attend R3
4	Write-back	Decode	Fetch	Instruction 2 reprend
5		Execute	Decode (bloqué, attend R5)	Instruction 3 attend R5
6		Write-back	Decode	Instruction 3 reprend
7			Execute	Instruction 3 en exécution
8			Write-back	Instruction 3 terminée

Cycle d'attente (stall) :

1. Cycle 3 :

Instruction 2 (Multiply) ne peut pas passer à l'étape Execute car R3 n'est pas encore produit par l'Instruction 1 (Write-back au Cycle 4).

Solution : Cycle d'attente inséré pour permettre la production de R3.

2. Cycle 5 :

Instruction 3 (Add) est bloquée à Decode, car R5 n'est pas encore disponible.

Solution : Cycle d'attente inséré pour attendre la production de R5

Langage machine

Langage machine

Le langage machine est constitué d'instructions directement compréhensibles par le processeur. Elles sont codées en binaire (suite de 0 et 1). Chaque processeur a son propre ensemble d'instructions (jeu d'instructions) et formats binaires associés.

Langage machine

Les caractéristiques sont:

- Difficile à lire et à écrire pour les humains.
- Très précis et adapté à la machine.
- Peu intuitif, mais efficace pour l'exécution.

Langage machine

Exemple d'instruction binaire :

Une instruction pour additionner deux registres :

1010 0001

Ici, 1010 représente l'opcode (l'opération d'addition).

0001 pourrait représenter les registres ou les données associées.

Conclusion

L'architecture des ensembles d'instructions (ISA) est au cœur du fonctionnement des processeurs. Elle définit l'interface entre le matériel (processeur) et le logiciel (programme). En spécifiant les commandes que le processeur peut comprendre et exécuter, l'ISA joue un rôle crucial dans la conception et l'efficacité des systèmes informatiques.

Chapitre 4:

Programmation niveau

assembleur

Langage bas niveau

Un langage de programmation bas niveau est un langage qui dépend fortement de la structure matérielle de la machine.

Cette caractéristique offre des avantages et des inconvénients divers.

Elle permet, entre autres,

- d'avoir un bon contrôle des **ressources matérielles** ;
- d'avoir un contrôle très fin sur la **mémoire** ;
- souvent, de produire du code dont l'exécution est très **rapide**.

En revanche, elle ne permet pas

- d'utiliser des techniques de programmation abstraites ;
- de programmer rapidement et facilement.

Langage machine

Un langage machine est un langage compris directement par le processeur en vue d'une exécution.

C'est un langage binaire : ses seules lettres sont les bits 0 et 1.

Chaque modèle de processeur possède son propre langage machine. Étant donnés deux modèles de processeurs P_1 et P_2 , on dit que P_1 est compatible avec P_2 si toute instruction formulée pour P_2 peut être comprise et exécutée par P_1 .

Dans la plupart des langages machine, une instruction commence par un opcode, une suite de bits qui porte la nature de l'instruction. Celui-ci est suivi des suites de bits codant les opérandes de l'instruction.

– Exemple –

La suite

01101010 00010101

est une instruction dont le opcode est 01101010 et l'opérande est 00010101. Elle ordonne de placer la valeur $(21)_{\text{dix}}$ en tête de la pile.

Langages d'assemblage

Un langage d'assemblage (ou assembleur) est un langage qui se trouve à mi-distance entre le programmeur et la machine en terme de facilité d'accès.

En effet, d'un côté, la machine peut convertir presque immédiatement un programme en assembleur vers du langage machine. De l'autre, l'assembleur est un langage assez facile à manipuler pour un humain.

Les opcodes sont codés via des mnémoniques, mots-clés bien plus manipulables pour le programmeur que les suites binaires associées.

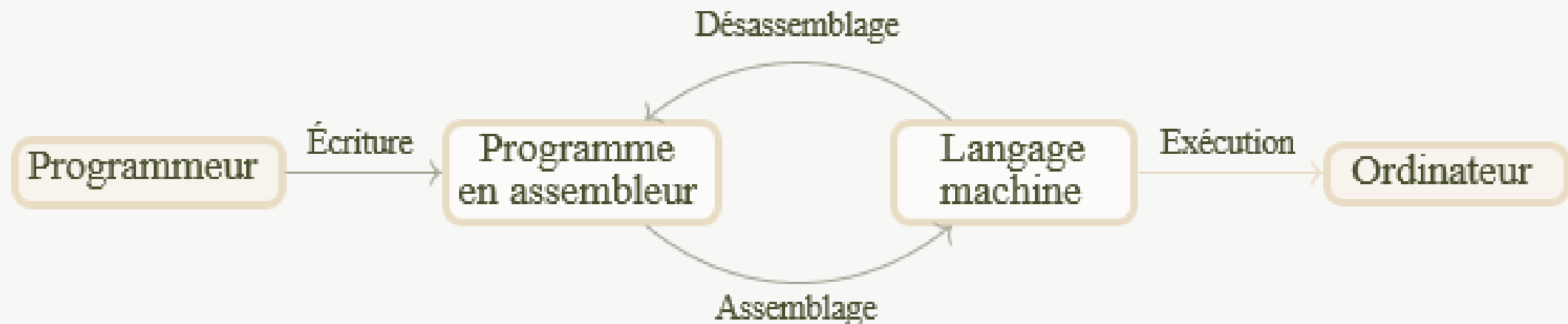
Du fait qu'un langage d'assemblage est spécifiquement dédié à un processeur donné, il existe presque autant de langages d'assemblage qu'il y a de modèles de processeurs.

Langages d'assemblage et assembleurs

L'assemblage est l'action d'un programme nommé **assembleur** qui consiste à traduire un programme en assembleur vers du langage machine.

Le désassemblage, réalisé par un **désassembleur**, est l'opération inverse. Elle permet de retrouver, à partir d'un programme en langage machine, un programme en assembleur qui lui correspond.

Voici le schéma opérationnel liant tout ceci :



L'assembleur NASM

Pour des raisons pédagogiques, nous choisissons de travailler sur une architecture **x86** en 32 bits. C'est une architecture dont le modèle de processeur est compatible avec le modèle Intel 8086.

Nous utiliserons l'**assembleur NASM** (Netwide Assembler).

Pour programmer, il faudra disposer :

1. d'un ordinateur (moderne);
2. d'un système en 32 bits (réel ou virtuel) ou 64 bits ;
3. d'un éditeur de textes ;
4. du programme `nasm` (assembleur) ;
5. du programme `ld` (lieur) ou `gcc` ;
6. du programme `gdb` (débugueur), non indispensable à ce stade.

Généralités

Un programme assembleur est un fichier texte d'extension .asm.

Il est constitué de plusieurs parties dont le rôle est

1. d'invoquer des **directives** ;
2. de définir des **données initialisées** ;
3. de réserver de la mémoire pour des **données non initialisées** ;
4. de contenir une suite **instructions**.

Nous allons étudier chacune de ces parties.

Avant cela, nous avons besoin de nous familiariser avec trois ingrédients de base dans la programmation assembleur : les **valeurs**, les **registres** et la **mémoire**.

Exprimer des valeurs

Il y a plusieurs manières d'exprimer des valeurs en assembleur.

Les **entiers** sont représentés en interne selon le codage en complément à deux. Ils s'écrivent

- directement en base dix, p.ex., `0` , `10020` , `-91` ;
- en hexadécimal, avec le préfixe `0x` , p.ex., `0xA109C` , `-0x98` ;
- en binaire, avec le préfixe `0b` , p.ex., `0b001` , `0b11101` .

Les **caractères** sont représentés en interne par leur code ASCII. Ils s'écrivent

- directement, p.ex., `'a'` , `'9'` ;
- par leur code ASCII, p.ex., `10` , `120` .

Les **chaînes de caractères** sont des suites de caractères. Elles s'écrivent

- directement, p.ex., `'abbaa'` , `0` ;
- caractère par caractère, p.ex., `'a'` , `46` , `36` , `0` .

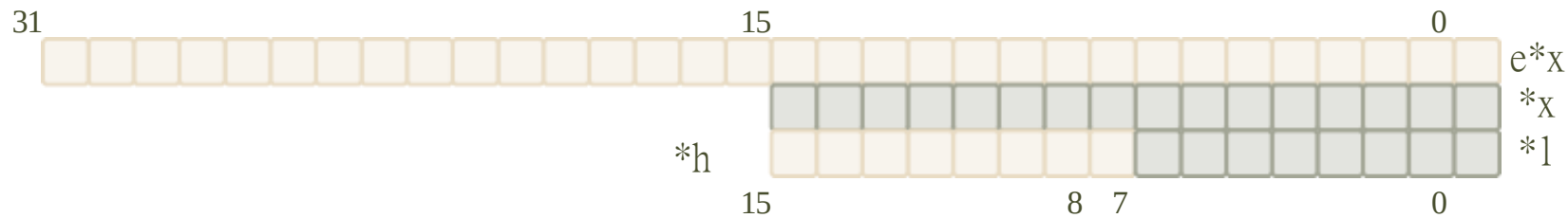
Le code ASCII du marqueur de fin de chaîne est `0` (à ne pas oublier).

Registres et sous-registres

Un registre est un emplacement de 32 bits.

On peut le considérer comme une **variable globale**.

Il y a quatre registres de travail : `eax` , `ebx` , `ecx` et `edx` . Il sont subdivisés en sous-registres selon le schéma suivant :



– Exemple –

`bh` désigne le 2^e octet de `ebx` et `cx` désigne les deux 1^{ers} octets de `ecx` .

Il est possible d'écrire / lire dans chacun de ces (sous-)registres.

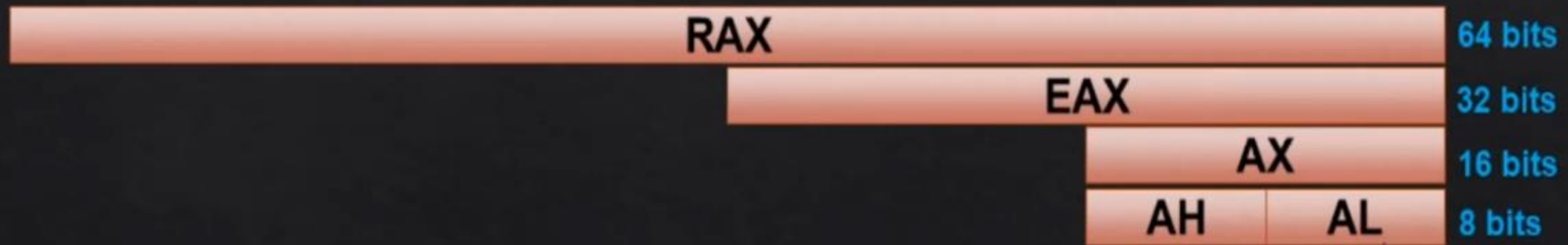
Attention : toute modification d'un sous-registre entraîne bien évidemment une modification du registre tout entier (et réciproquement).

Registres et sous-registres

En 16 bits on a :

- ✧ AX = Accumulator → On y stock souvent un résultat ou une valeur qui sera lue comme input d'une fonction)
- ✧ BX = Base → Sa valeur est souvent une « adresse mémoire »
- ✧ CX = Counter → Un compteur, sera souvent incrémenté
- ✧ DX = Data → Manipulation de données

On vrai on peut s'en servir comme on veut. Autres représentations : (exemple avec AX)



Registres de diverses sortes

Il existe d'autres registres. Parmi ceux-ci, il y a

- le pointeur d'instruction `eip`, contenant l'adresse de la prochaine instruction à exécuter;
- le pointeur de tête de pile `esp`, contenant l'adresse de la tête de la pile;
- le pointeur de pile `ebp`, utilisé pour contenir l'adresse d'une donnée de la pile dans les fonctions;
- le registre de drapeaux `flags`, utilisé pour contenir des informations sur le résultat d'une opération qui vient d'être réalisée.

Attention : ce ne sont pas des registres de travail, leurs rôles sont fixés. Il est possible pour certains de ces registres d'y écrire ou lire des valeurs explicitement.

Instructions basiques

Instructions basiques

Instructions		Exemples		Résultats
◇ MOV [destination], [source]	→	MOV AX, 1337	→	AX = 1337
◇ ADD [destination], [source]	→	ADD AX, 3	→	AX = 1340
◇ SUB [destination], [source]	→	SUB AX, 340	→	AX = 1000
◇ INC [destination]	→	INC AX	→	AX = 1001
◇ DEC [destination]	→	DEC AX	→	AX = 1000
◇ NEG [destination]	→	NEG AX	→	AX = -1000

◇ Il y a aussi MUL (multiplication), DIV (division) et plein d'autres

L'opération mov — définition

L'instruction

```
mov REG, VAL
```

permet de **recopier** la valeur **VAL** dans le (sous-)registre **REG**.

— Exemples —

Voici les effets de quelques instructions :

■ `mov eax, 0` `eax =`

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

■ `mov ebx, 0xFFFFFFFF` `ebx =`

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

■ `mov eax, 0b101` `eax =`

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

■ `mov al, 5` `eax =`

*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 où le
symbole * dénote une valeur non modifiée.

L'opération mov — respect des tailles

Il est important, pour que l'instruction `mov REG, VAL` soit correcte, que la taille en octets de la valeur `VAL` soit la même que celle du (sous-)registre `REG`.

– Exemples –

Les instructions suivantes **ne sont pas correctes** :

- `mov al, 0xA5A5` Le sous-registre `al` occupe 1 octet alors que la valeur `0xA5A5` en occupe 2.
- `mov ax, ebx` Le sous-registre `ax` occupe 2 octets alors que la valeur contenue dans `ebx` en occupe 4.
- `mov eax, cx` Le sous-registre `eax` occupe 4 octets alors que la valeur contenue dans `cx` en occupe que 2.

– Exemples –

En revanche, les instructions suivantes **sont correctes** :

- `mov al, 0xA5` La valeur `0xA5` occupe bien 1 octet, tout comme `al`.
- `mov ax, 0xF` Le sous-registre `ax` occupe 2 octets alors que la valeur `0xF` n'en occupe que 1. Néanmoins, cette valeur est étendue sur 2 octets sans perte d'information en `0x000F`.

Opérations sur les registres — arithmétique

Opérations **arithmétiques** :

- `add REG, VAL` incrémente le (sous-)registre `REG` de la valeur `VAL` ;
- `sub REG, VAL` décrémente le (sous-)registre `REG` de la valeur `VAL` ;
- `mul REG` multiplie la valeur contenue dans `eax` et le (sous-)registre `REG` et place le résultat dans `edx:eax`. Cette notation désigne la suite de 64 bits dont les bits de `edx` sont ceux de poids forts et ceux de `eax` ceux de poids faibles);
- `div REG` place le quotient de la division de `edx:eax` par la valeur portée par le (sous-)registre `REG` dans `eax` et le reste dans `edx`.

– Exemple –

```
mov eax, 20 ;eax = 20
add eax, 51 ;eax = 71
add eax, eax ;eax = 142
```

```
mov ebx, 3 ;ebx = 3
mul ebx ;eax = 426
```

```
mov edx, 0 ;edx = 0
sub ebx, 1 ;ebx = 2
div ebx ;eax = 213
```

Opérations sur les registres — logique

Opérations **logiques** :

- `not REG` place dans le (sous-)registre `REG` la valeur obtenue en réalisant le *non* bit à bit de sa valeur;
- `and REG, VAL` place dans le (sous-)registre `REG` la valeur du *et* logique bit à bit entre les suites contenues dans `REG` et `VAL` ;
- `or REG, VAL` place dans le (sous-)registre `REG` la valeur du *ou* logique bit à bit entre les suites contenues dans `REG` et `VAL` ;
- `xor REG, VAL` place dans le (sous-)registre `REG` la valeur du *ou exclusif* logique bit à bit entre les suites contenues dans `REG` et `VAL` .

Opérations sur les registres — bit à bit

Opérations **bit à bit** :

- `shl REG, NB` décale les bits du (sous-)registre `REG` à gauche de `NB` places et complète à droite par des `0` ;
- `shr REG, NB` décale les bits du (sous-)registre `REG` à droite de `NB` places et complète à gauche par des `0` ;
- `rol REG, NB` réalise une rotation des bits du (sous-)registre `REG` à gauche de `NB` places ;
- `ror REG, NB` réalise une rotation des bits du (sous-)registre `REG` à droite de `NB` places.

Instructions Logiques

◆ NOT		Effectue un complément à 1 (sur la valeur en binaire)
◆ OR		OU logique
◆ AND		ET logique
◆ XOR		OU exclusif (exemple : XOR EAX, EAX → EAX = 0)

Et plein d'autres (NAND, NOR, ...)

Instructions Spéciales

Instructions

- ◆ NOP → No OPeration (ne rien faire)
- ◆ JMP EBX → Jump. Sauter vers une adresse mémoire, ici la valeur d'EBX)
- ◆ CALL EBX → Appeler une procédure. Equivaut à sauter vers une adresse mémoire, ici la valeur d'EBX. Mais revenir au flux initial après
- ◆ CMP EDI, ESI → Comparer ESI et EDI
Retourne 0 s'il y a 0 différence. Donc le ZF passe à 1
- ◆ PUSH 0x42 → Pousse la valeur 0x42 sur le sommet de la pile
- ◆ POP EAX → Récupère la valeur du sommet de la pile dans EAX : EAX = 0x42
Et déplace ESP (Stack Pointer) d'un cran en-dessous
- ◆ INT 0x80 → Interruption système (interagir avec le hardware de l'OS)

Mémoire

Les registres n'offrent pas assez de mémoire pour construire des programmes élaborés. C'est pour cette raison que l'on utilise la **mémoire**.

La mémoire est segmentée en plusieurs parties :

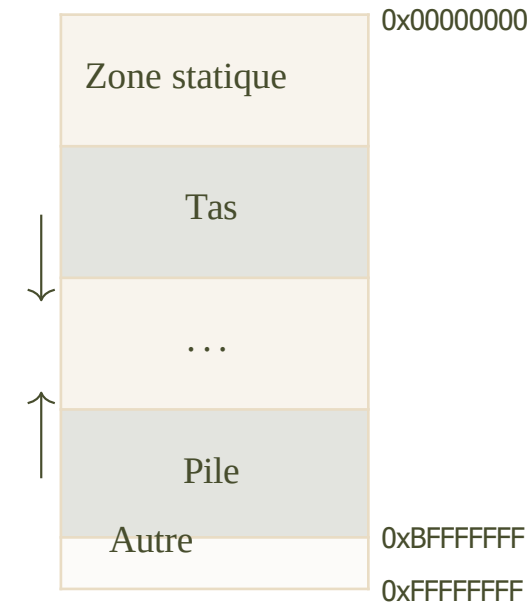
- la zone statique qui contient le code et les données statiques ;
- le tas, de taille variable au fil de l'exécution;
- la pile, de taille variable au fil de l'exécution.

Il y a d'autres zones (non représentées ici).

En mode protégé, chaque programme en exécution possède son propre environnement de mémoire. Les adresses y sont relatives et non absolues.

De cette manière, un programme en exécution ne peut empiéter sur la mémoire d'un autre.

La lecture / écriture en mémoire suit la convention *little-endian*.



Lecture en mémoire

L'instruction

```
mov REG, [ADR]
```

place dans le (sous-)registre **REG** un, deux ou quatre octets en fonction de la taille de **REG**, **lus** à partir de l'adresse **ADR** **dans la mémoire**.

En supposant que **x** soit une adresse accessible en mémoire, les parties foncées sont celles qui sont lues et placées dans le (sous-)registre opérande de l'instruction :

x	*	*	*	*	*	*	*	*
x + 1	*	*	*	*	*	*	*	*
x + 2	*	*	*	*	*	*	*	*
x + 3	*	*	*	*	*	*	*	*

```
mov ah, [x] ou mov al, [x]
```

x	*	*	*	*	*	*	*	*
x + 1	*	*	*	*	*	*	*	*
x + 2	*	*	*	*	*	*	*	*
x + 3	*	*	*	*	*	*	*	*

```
mov ax, [x]
```

x	*	*	*	*	*	*	*	*
x + 1	*	*	*	*	*	*	*	*
x + 2	*	*	*	*	*	*	*	*
x + 3	*	*	*	*	*	*	*	*

```
mov eax, [x]
```

Lecture en mémoire

– Exemple –

Observons l'effet des instructions suivantes sur `eax` avec la mémoire dans l'état suivant :

x	1	0	0	0	0	0	0	1
x + 1	1	1	1	1	1	1	1	1
x + 2	1	1	0	0	0	0	1	1
x + 3	1	0	1	0	1	0	1	0

```
mov eax, 0
```

```
mov al, [x]
```

```
mov ah, [x + 1]
```

```
mov ax, [x]
```

```
mov eax, [x]
```

```
mov ah, [x + 2]
```

```
mov ax, [x + 2]
```

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

01 0 10 0 01 0 10 0 01 10 0 0 0 0 10

01 0 10 0 10 0 10 0 10 10 0 0 0 0 01

10 0 10 0 01 0 01 0 10 10 0 0 0 0 10
10

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1

0 10 10 10 10 10 10 10 10 10 0 0 0 0 0 0 0 01

0 10 10 10 10 10 10 10 10 10 00 00 00 00 00 00 10

01 10 10 10 01 10 01 10 10 01 0 0 0 0 0 0 10

01 10 01 01 10 00 10 10 10 10 0 0 0 0 0 0 01

01 01 01 01 10 10 00 10 00 10 01 0 0 0 0 10

Subtilités de Syntaxe

- La syntaxe dépend de l'architecture du processeur / CPU :
 - **x86 ? x64 ? x16 ? PowerPC ? ARM ?**
- Il en existe 2 majeures:
- Processeurs Intel : **mov eax, ebx** Le plus commun
- Processeurs AT&T: **movl %ebx, %eax**
- Pour compliquer le tout, l'algorithme assembleur va varier en fonction du système d'exploitation (Windows/Linux/...) notamment pour les appels systèmes qui sont différents

EXAMPLE

MOV CX, 0

MOV BX, 10

XOR BX, BX

boucle:

INC CX

CMP CX, 10

JA fin

ADD BX, CX

JMP boucle

fin:

ret

FIN DU COURS ARCHITECTURE 2 DES ORDINATEURS

Pr ADG
Dr L. YADE
M. DIOP
M. ANNE